

Generative AI using Large Language Models (LLMs)

Lecture 7 | 03/05/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall



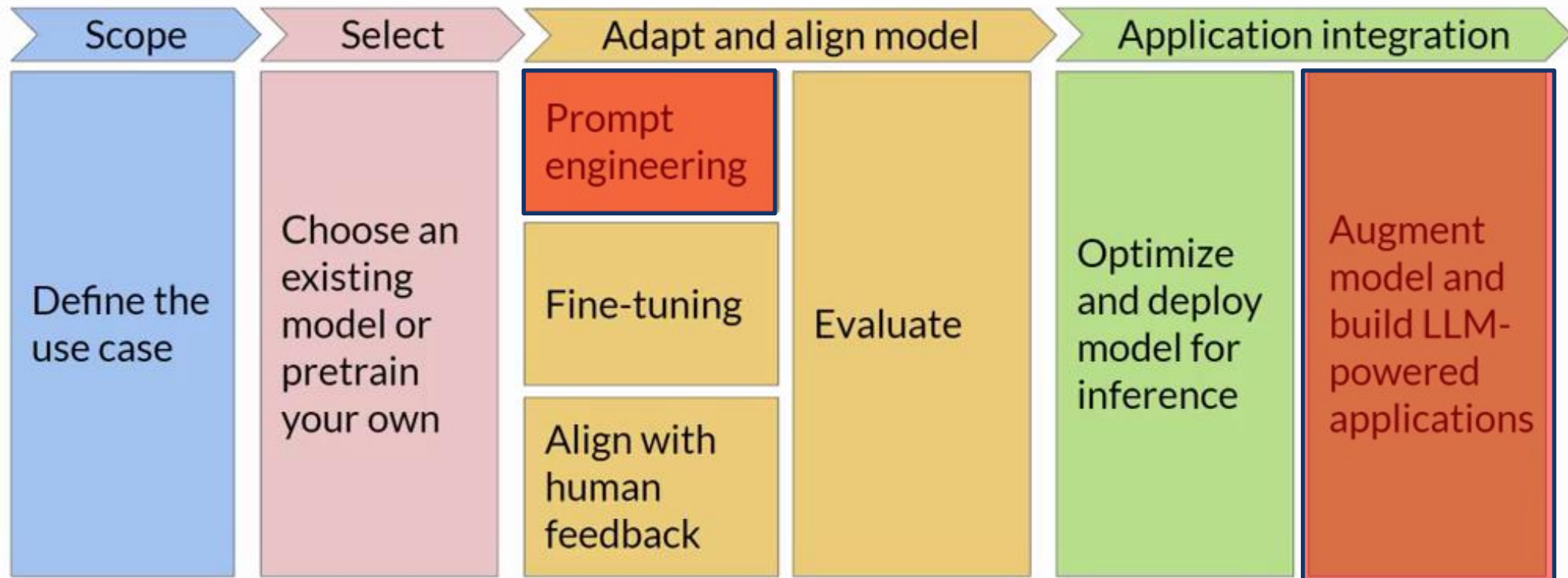
Agenda

- Pretrained Model Selection and Model Pretraining
- Quantization
- Efficient Mult-GPU Compute Strategies
- Scaling Law
- BloombergGPT



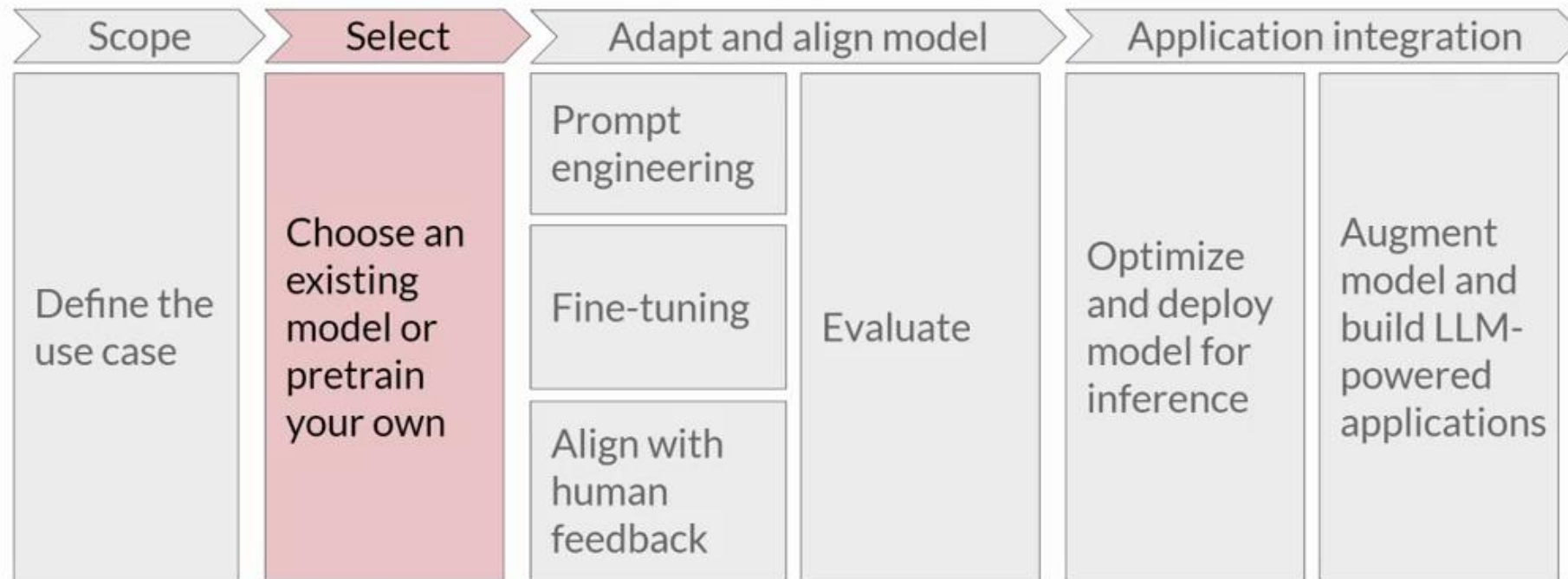


Generative AI project lifecycle





Generative AI project lifecycle





Considerations for choosing a model

Foundation model

Pretrained
LLM

Train your own model

Custom
LLM



Model Hubs

Model Card for T5 Large

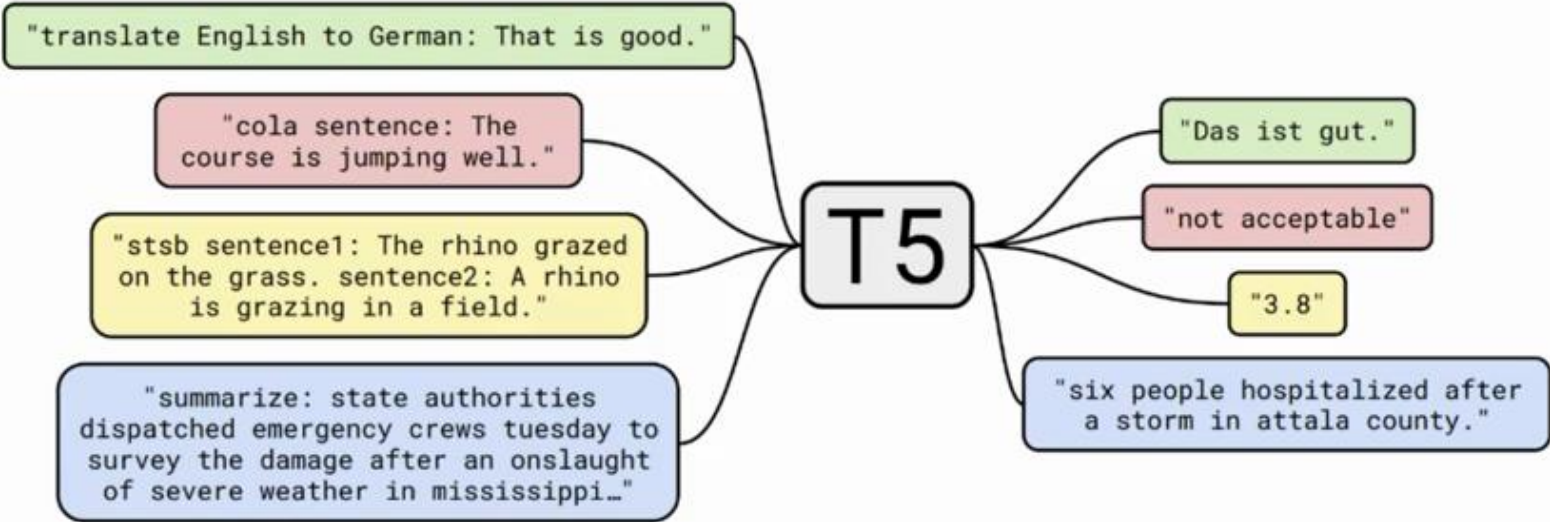


Table of Contents

- 1. [Model Details](#)
- 2. [Uses](#)
- 3. [Bias, Risks, and Limitations](#)
- 4. [Training Details](#)
- 5. [Evaluation](#)



How Large Language Models are trained?

Foundation model



Train your own model

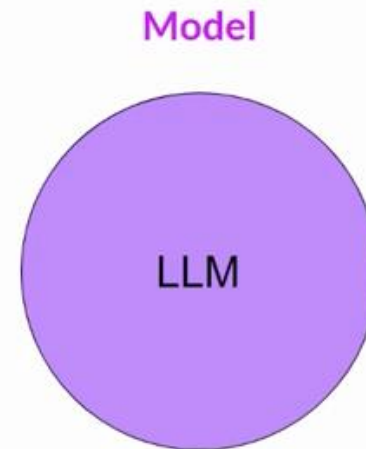


Model architectures and pre-training objectives



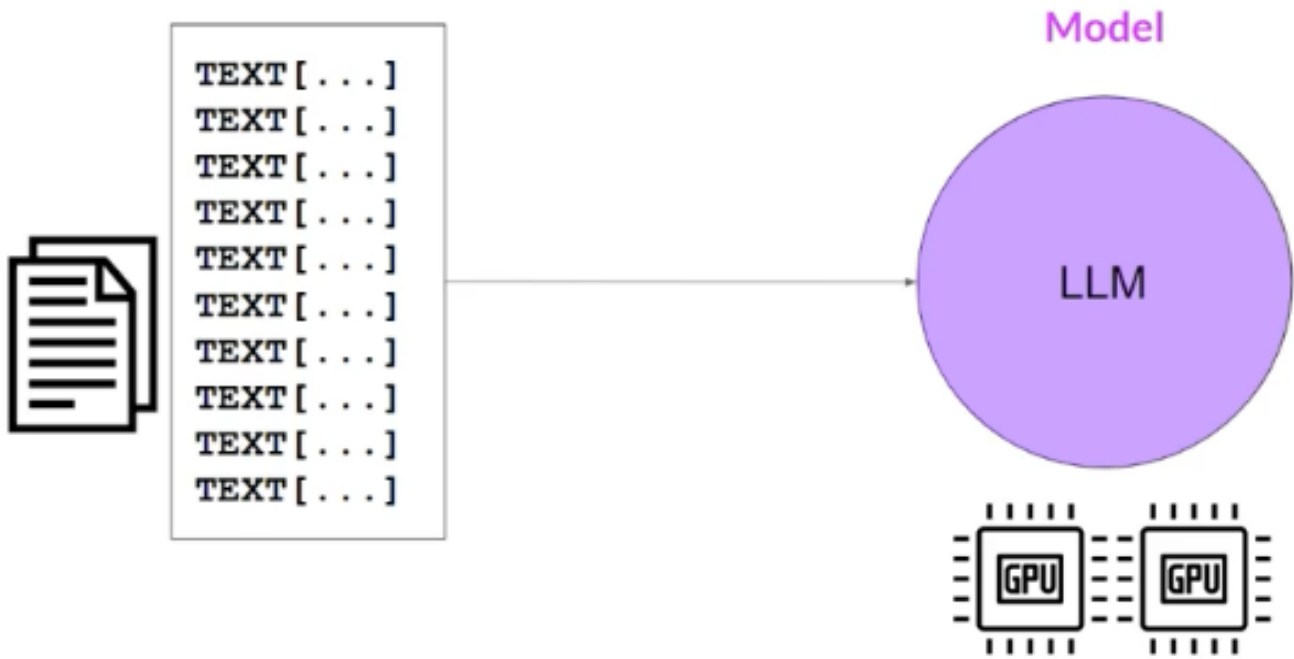


LLM pre-training at a high level





LLM pre-training at a high level



GB - TB - PB
of unstructured data

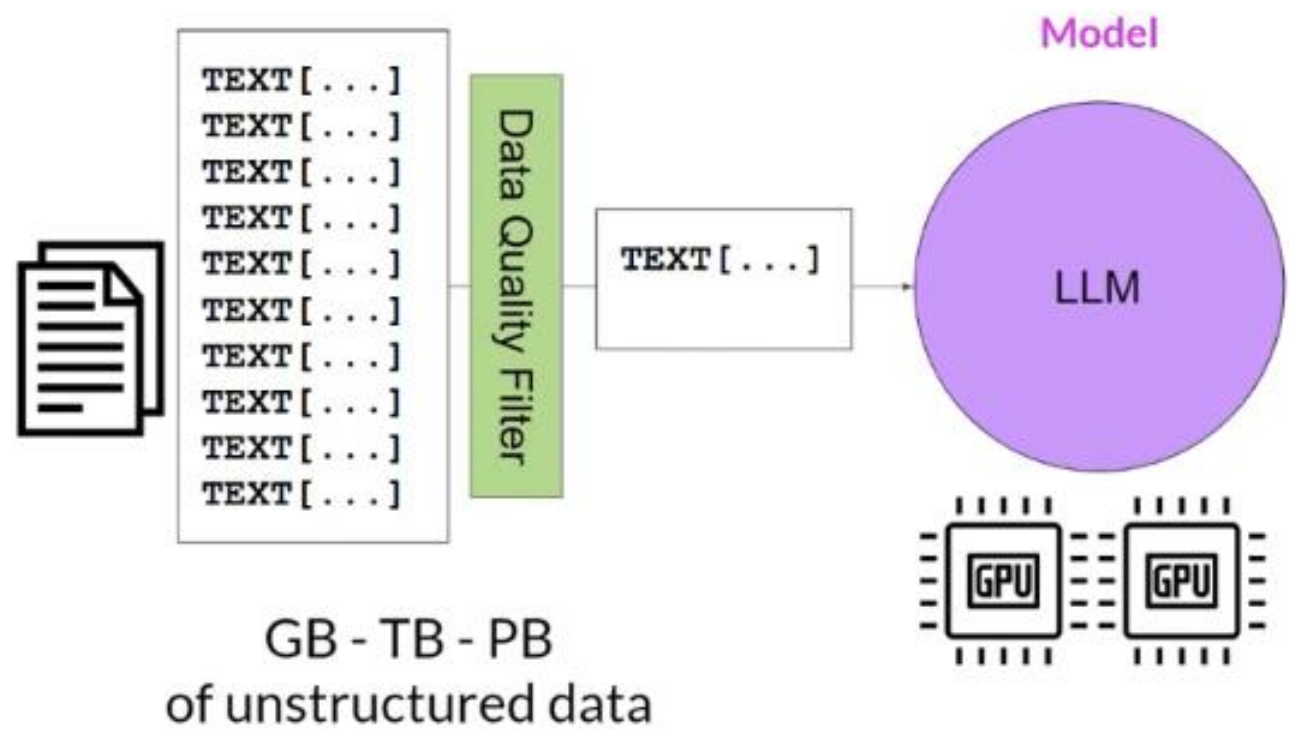
The model internalizes the patterns and structures
present in the language.

Token String	Token ID	Embedding / Vector Representation
'_The'	37	[-0.0513, -0.0584, 0.0230, ...]
'_teacher'	3145	[-0.0335, 0.0167, 0.0484, ...]
'_teaches'	11749	[-0.0151, -0.0516, 0.0309, ...]
'_the'	8	[-0.0498, -0.0428, 0.0275, ...]
'_student'	1236	[-0.0460, 0.0031, 0.0545, ...]
...	...	...

Vocabulary



LLM pre-training at high level

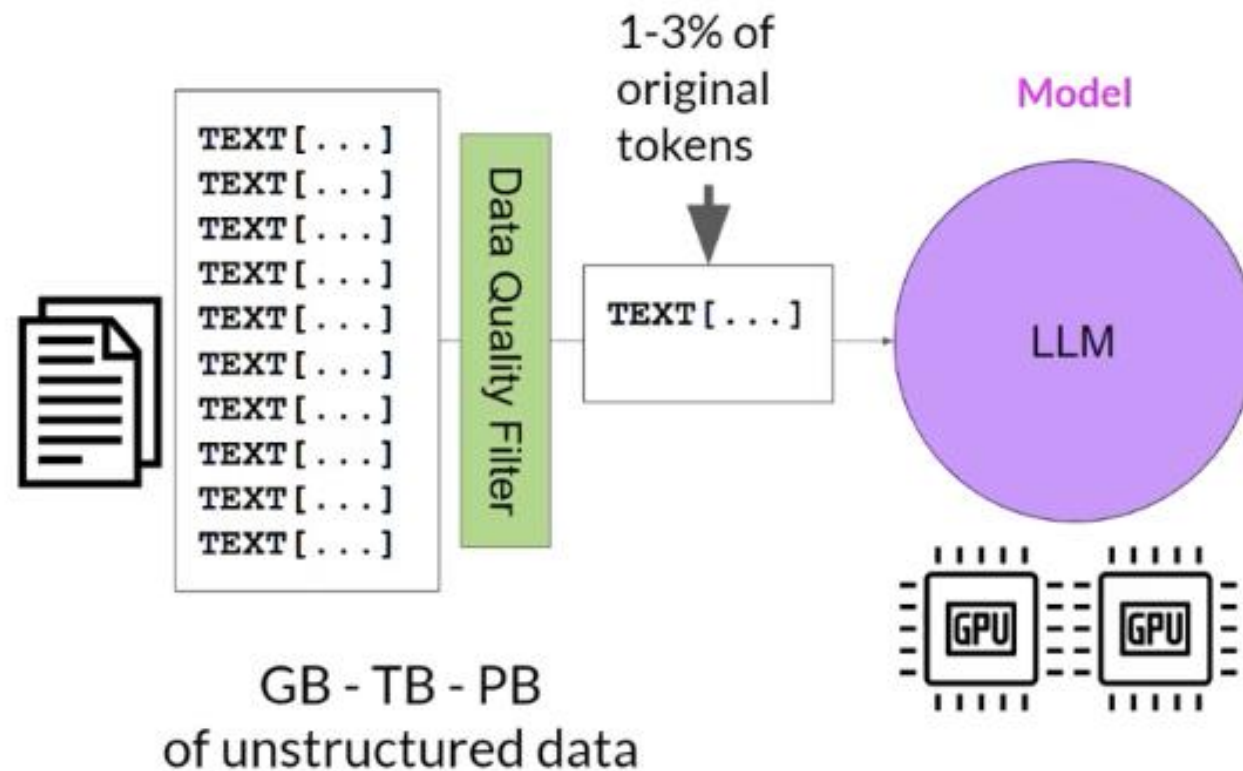


Token String	Token ID	Embedding / Vector Representation
'_The'	37	[-0.0513, -0.0584, 0.0230, ...]
'_teacher'	3145	[-0.0335, 0.0167, 0.0484, ...]
'_teaches'	11749	[-0.0151, -0.0516, 0.0309, ...]
'_the'	8	[-0.0498, -0.0428, 0.0275, ...]
'_student'	1236	[-0.0460, 0.0031, 0.0545, ...]
...	...	...

Vocabulary



LLM pre-training at high level

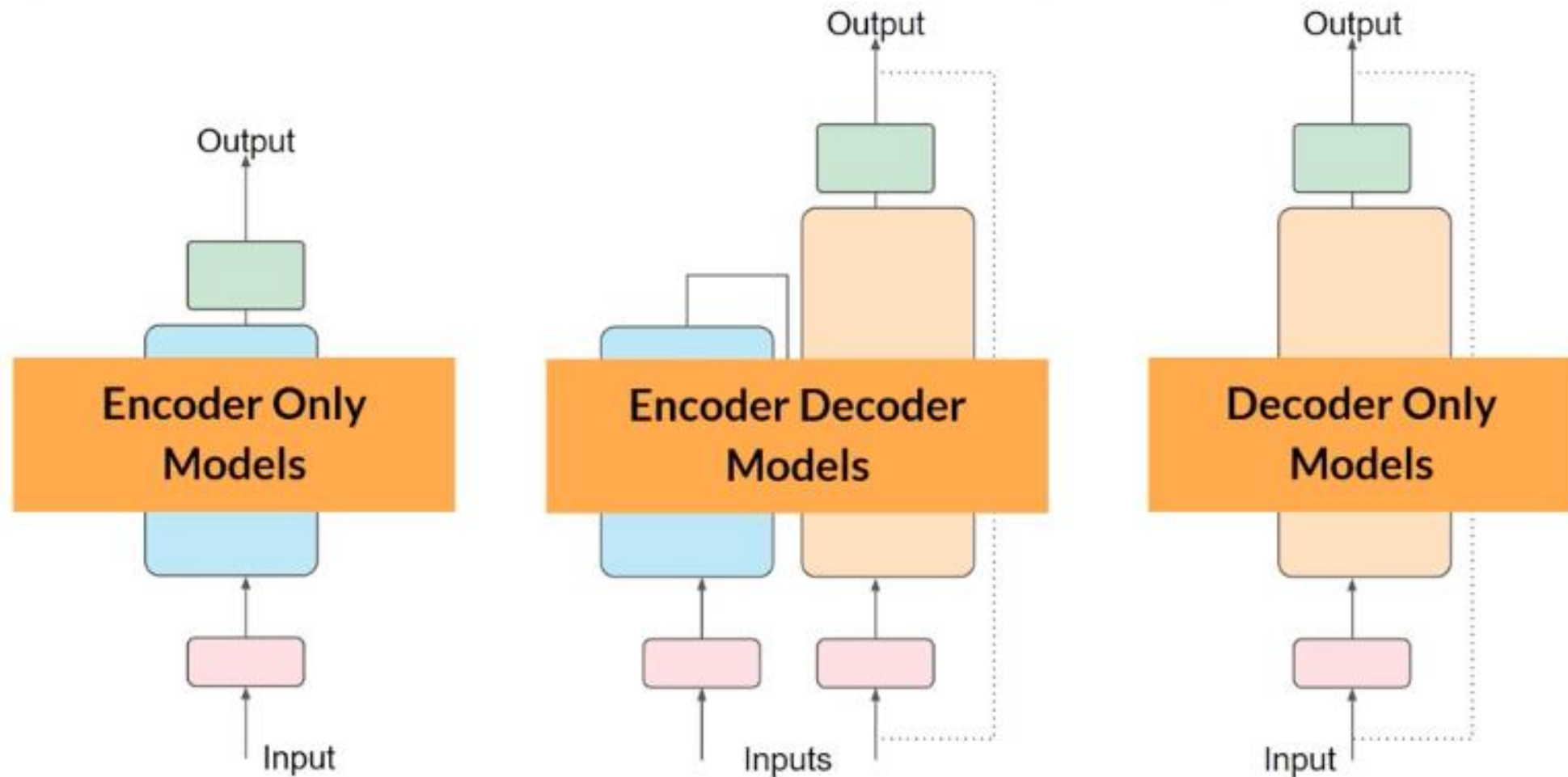


Token String	Token ID	Embedding / Vector Representation
'_The'	37	[-0.0513, -0.0584, 0.0230, ...]
'_teacher'	3145	[-0.0335, 0.0167, 0.0484, ...]
'_teaches'	11749	[-0.0151, -0.0516, 0.0309, ...]
'_the'	8	[-0.0498, -0.0428, 0.0275, ...]
'_student'	1236	[-0.0460, 0.0031, 0.0545, ...]
...	...	...

Vocabulary



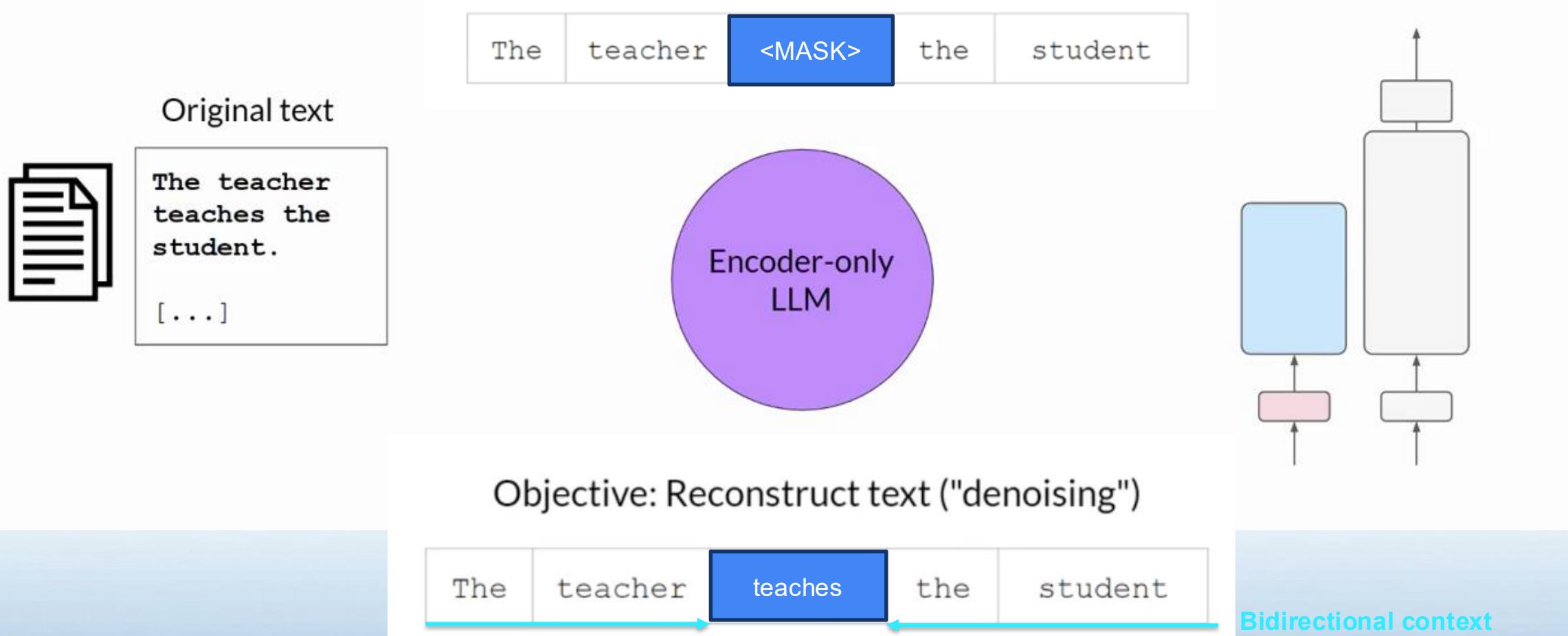
Transformers





Autoencoding models

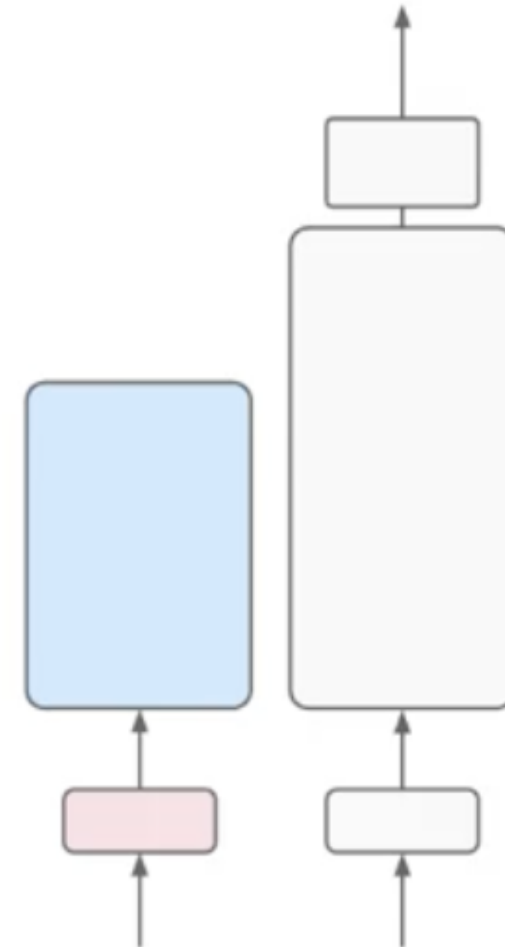
Masked Language Modeling (MLM)





Autoencoding models

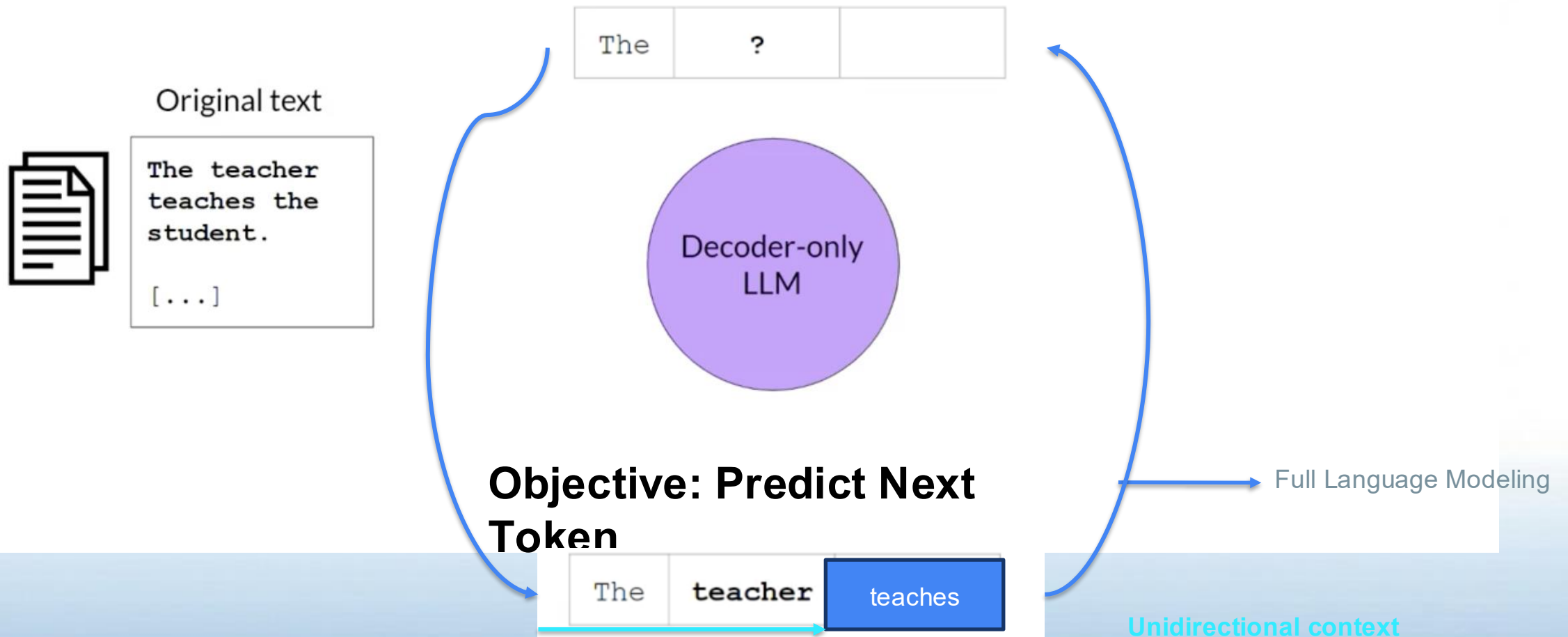
- Good use cases
 - Sentiment Analysis
 - Named entity recognition
 - Word classification
- Example models:
 - BERT
 - ROBERTA

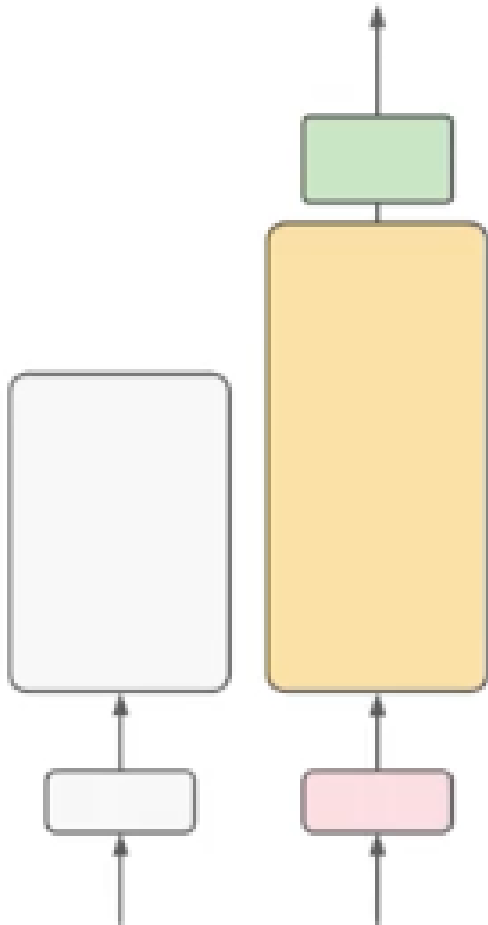




Autoregressive Models

Causal Language Modeling (CLM)





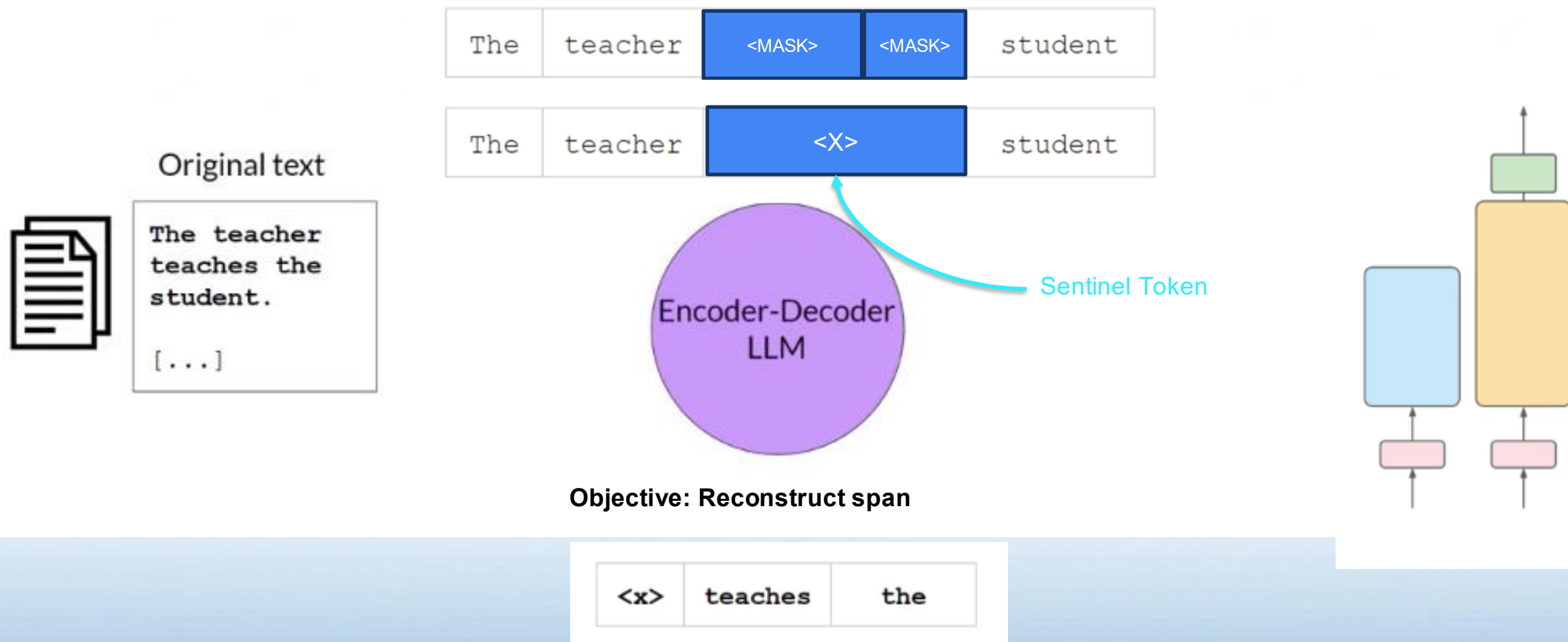
Autoregressive models

- Good use cases:
 - Text generation
 - Other emergent behavior
 - Depends on model size
- Example models:
 - GPT
 - BLOOM



Sequence-to-sequence models

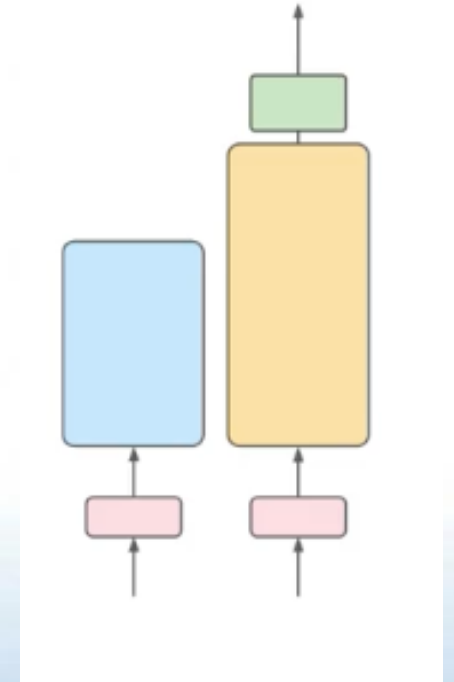
Span Corruption





Sequence-to-sequence models

- Good use cases:
 - Translation
 - Text Summarization
 - Question answering
- Example models
 - T5
 - BART





Model architectures and pre-training objectives

Target

Original text

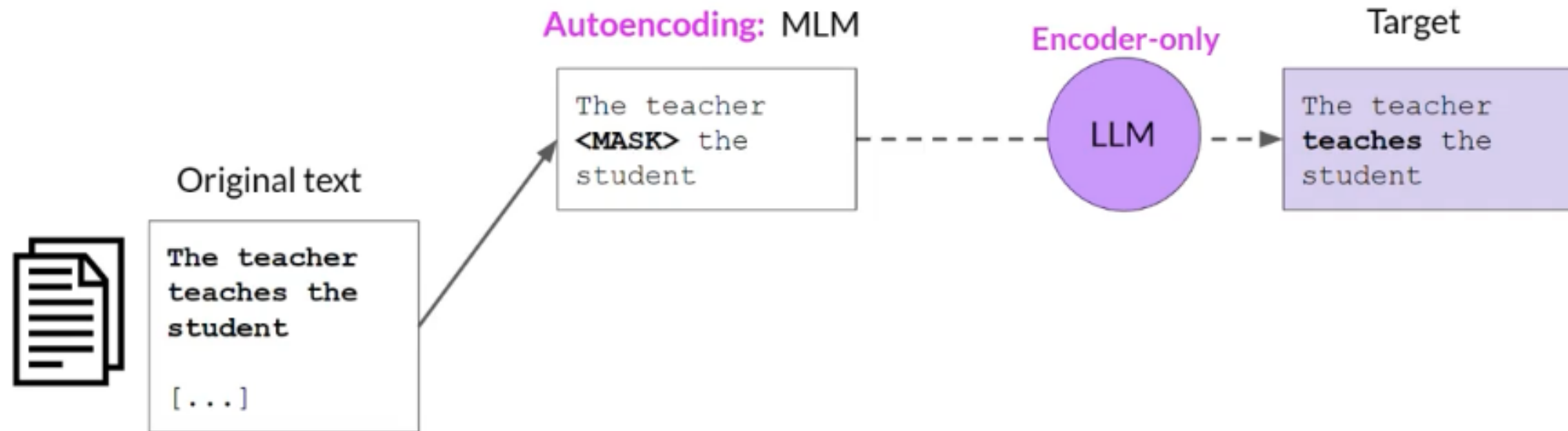


The teacher
teaches the
student

[...]

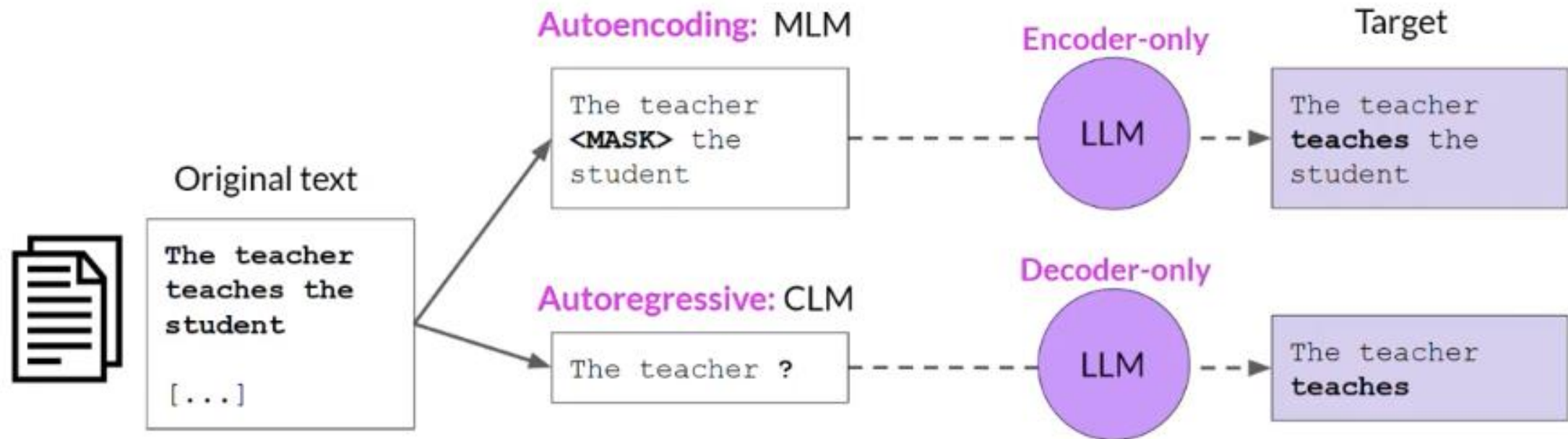


Model architectures and pre-training objectives



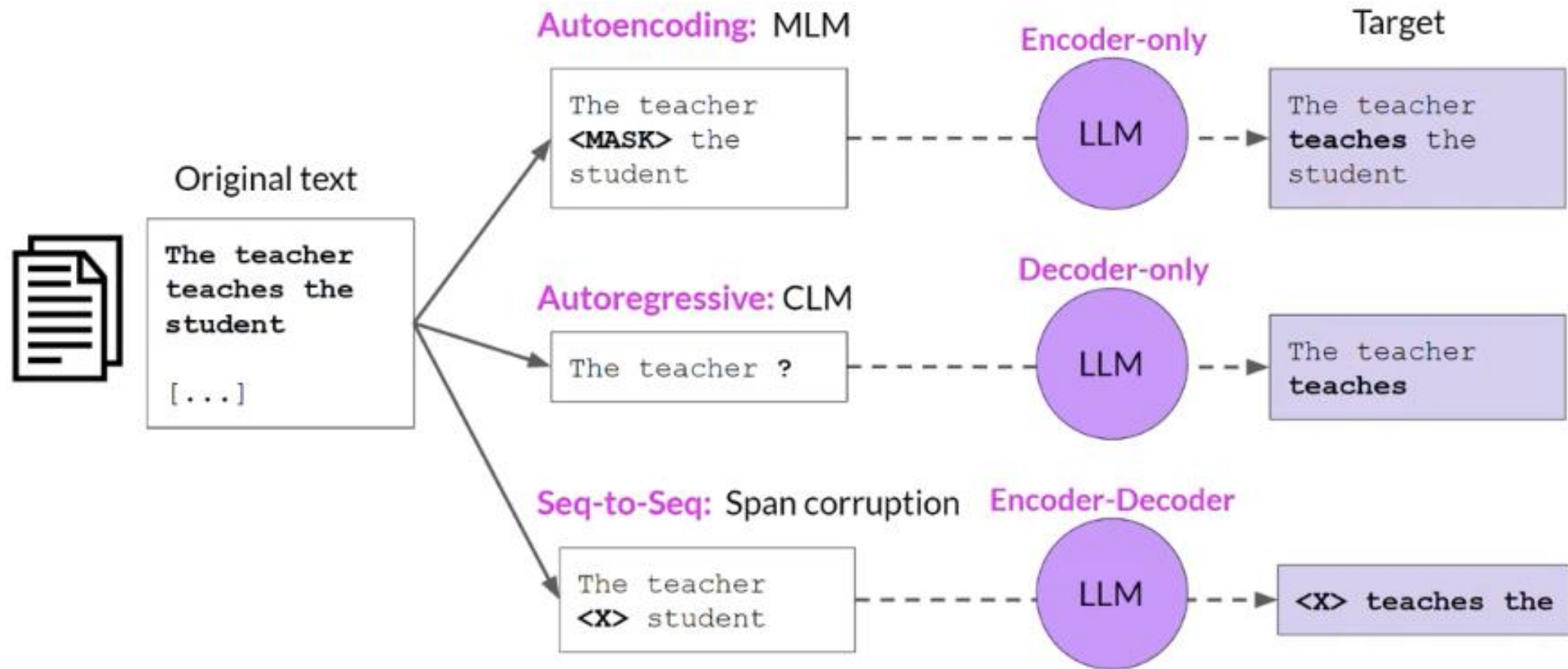


Model architectures and pre-training objectives





Model architectures and pre-training objectives





Question?

Large Language Models (LLMs) are capable of performing multiple tasks supporting a variety of use cases. Which of the following tasks supports the use case of converting code comments into executable code?

- ☒ Translation
- ☐ Invoke actions from text
- ☐ Information Retrieval
- ☐ Text summarization



Question?

Which transformer-based model architecture is well-suited to the task of text translation?

- Autoencoder
- Sequence-to-sequence
- Autoregressive

The significance of scale: task ability



BLOOM
176B



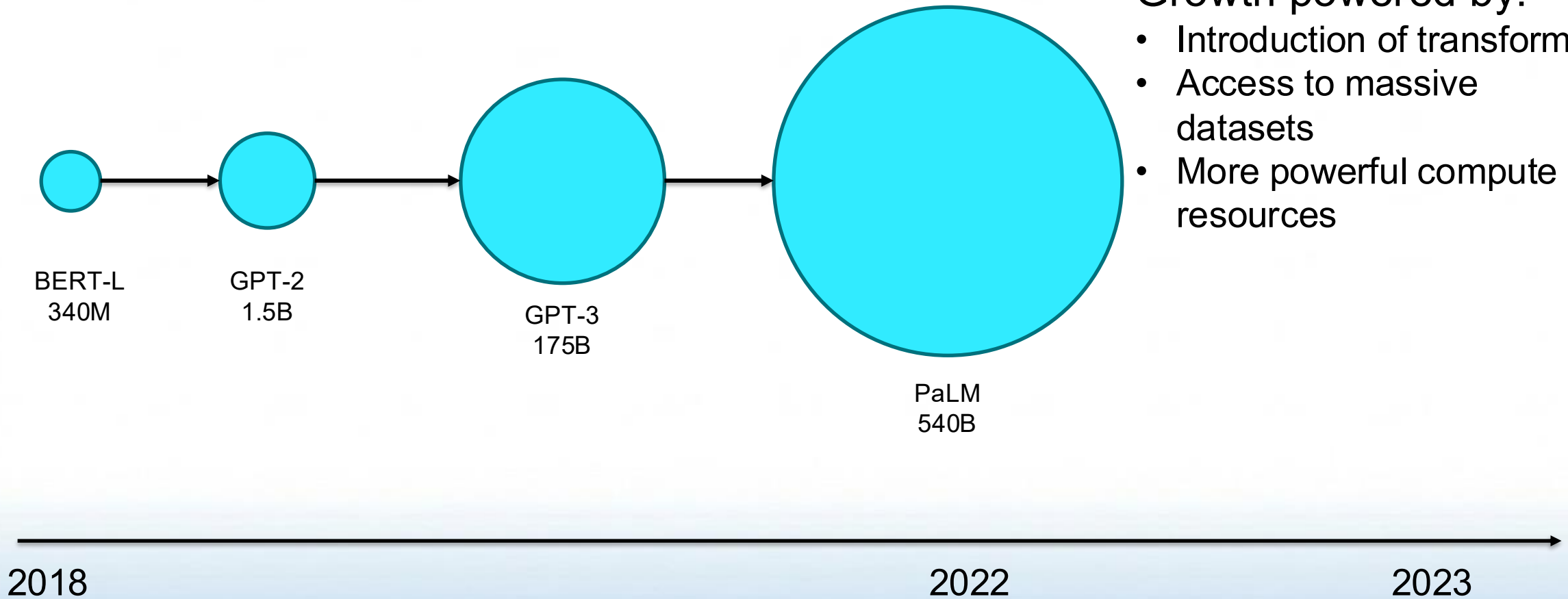
*Bert-base



Model size vs. time

Growth powered by:

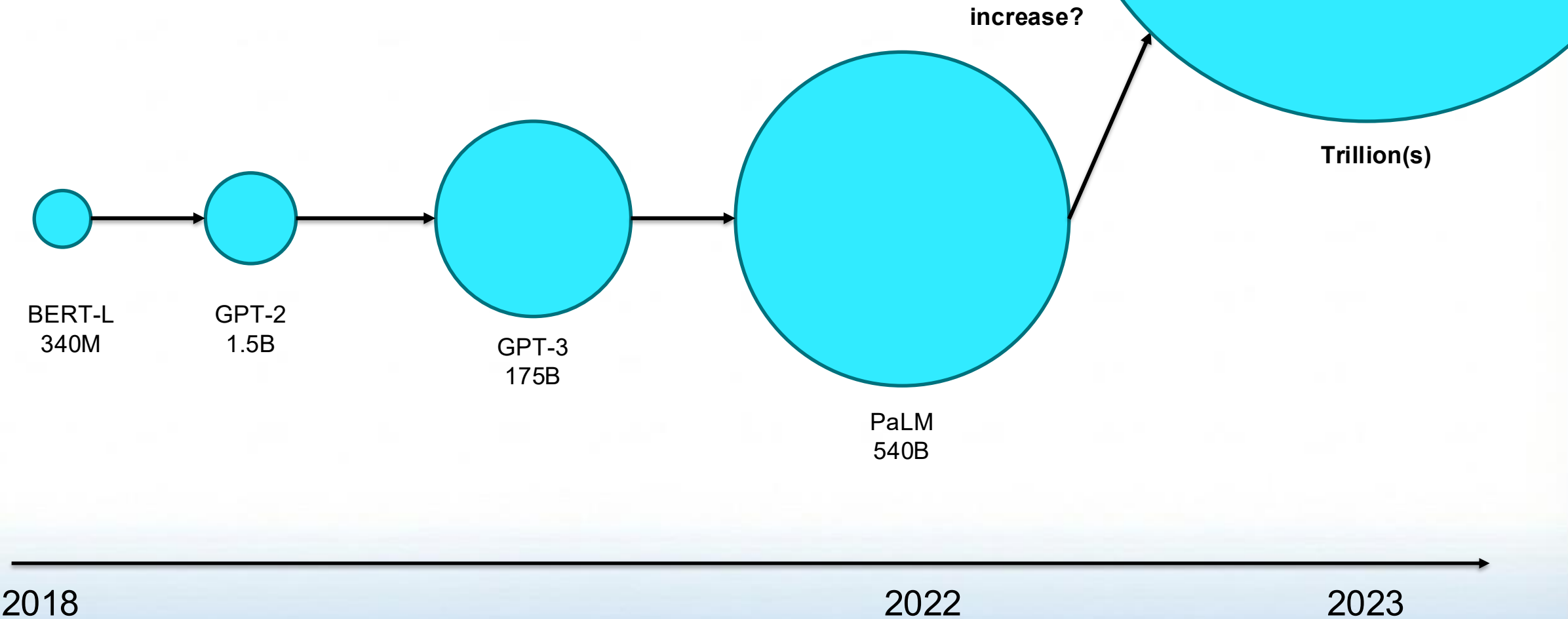
- Introduction of transformer
- Access to massive datasets
- More powerful compute resources





STA

Model size vs. time





Computational Challenges

`OutOfMemoryError: CUDA out of memory.`





Approximate GPU RAM needed to store 1B parameters

1 parameter = 4 bytes (32-bit float)

1B parameters = 4×10^9 bytes = 4 GB

4GB @ 32-bit
Full precision

References: <https://github.com/bitsandbytes-foundation/bitsandbytes>
https://huggingface.co/docs/transformers/v4.20.1/en/perf_train_gpu_one#anatomy-of-models-memory



Additional GPU RAM needed to train 1B parameters



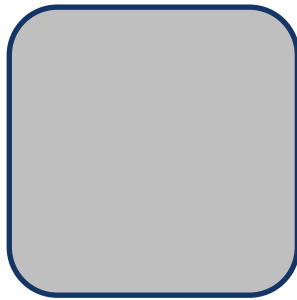
~ 20 extra bytes
per parameter

References: <https://github.com/bitsandbytes-foundation/bitsandbytes>
https://huggingface.co/docs/transformers/v4.20.1/en/perf_train_gpu_one#anatomy-of-models-memory



Approximate GPU RAM needed to train 1B-params

Memory needed to store model



4GB@32-bit
full precision

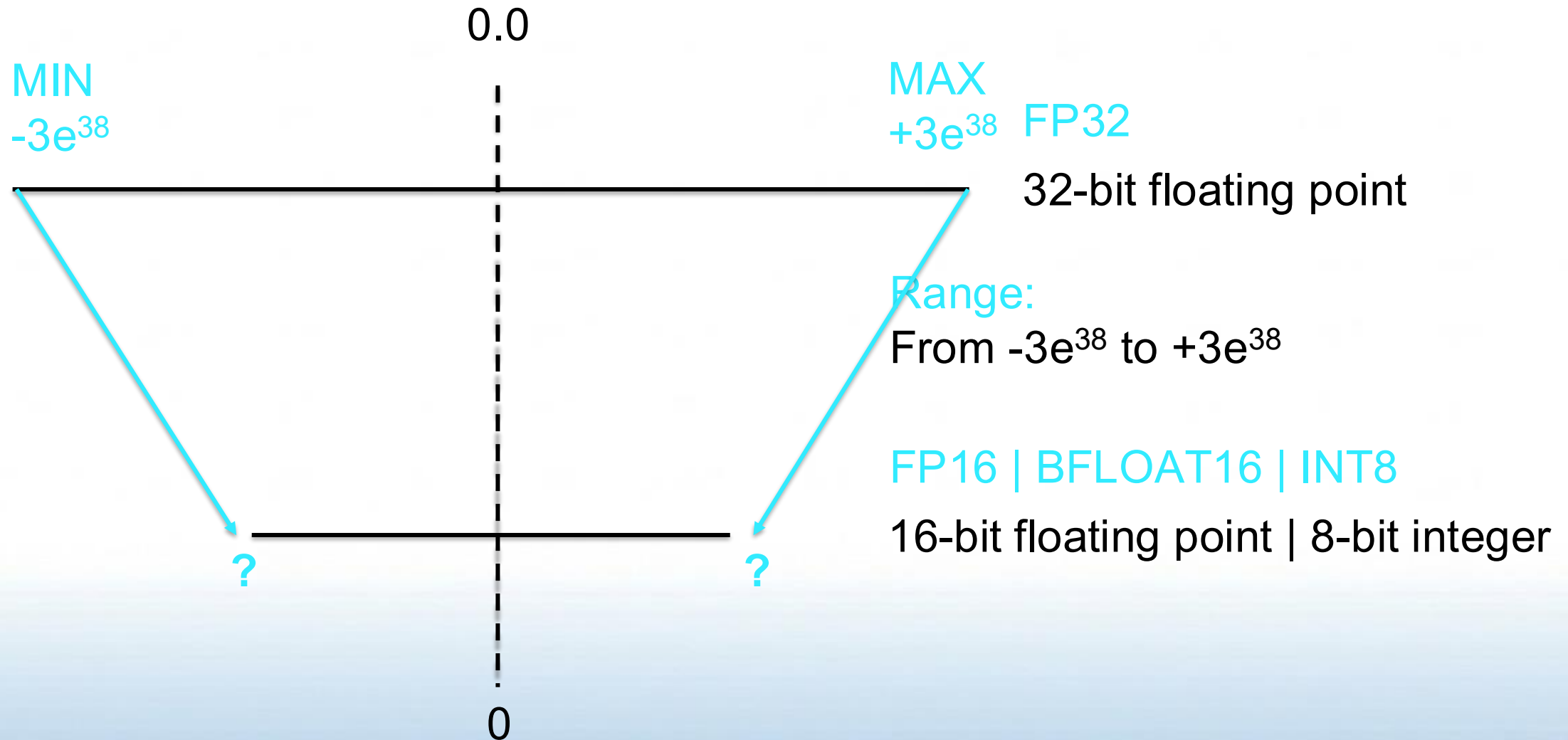
Memory needed to train model



24 GB @ 32-bit
full precision



Quantization

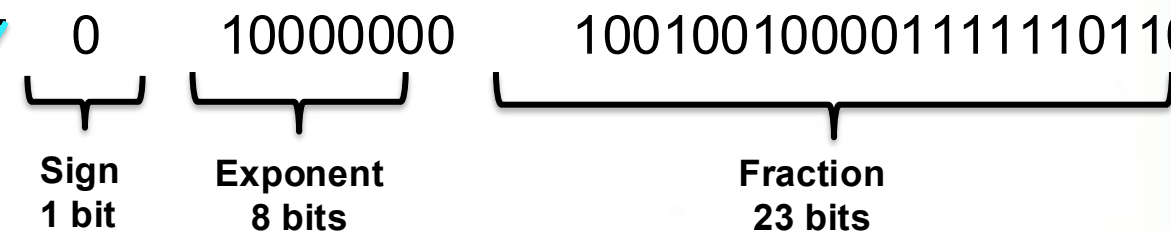
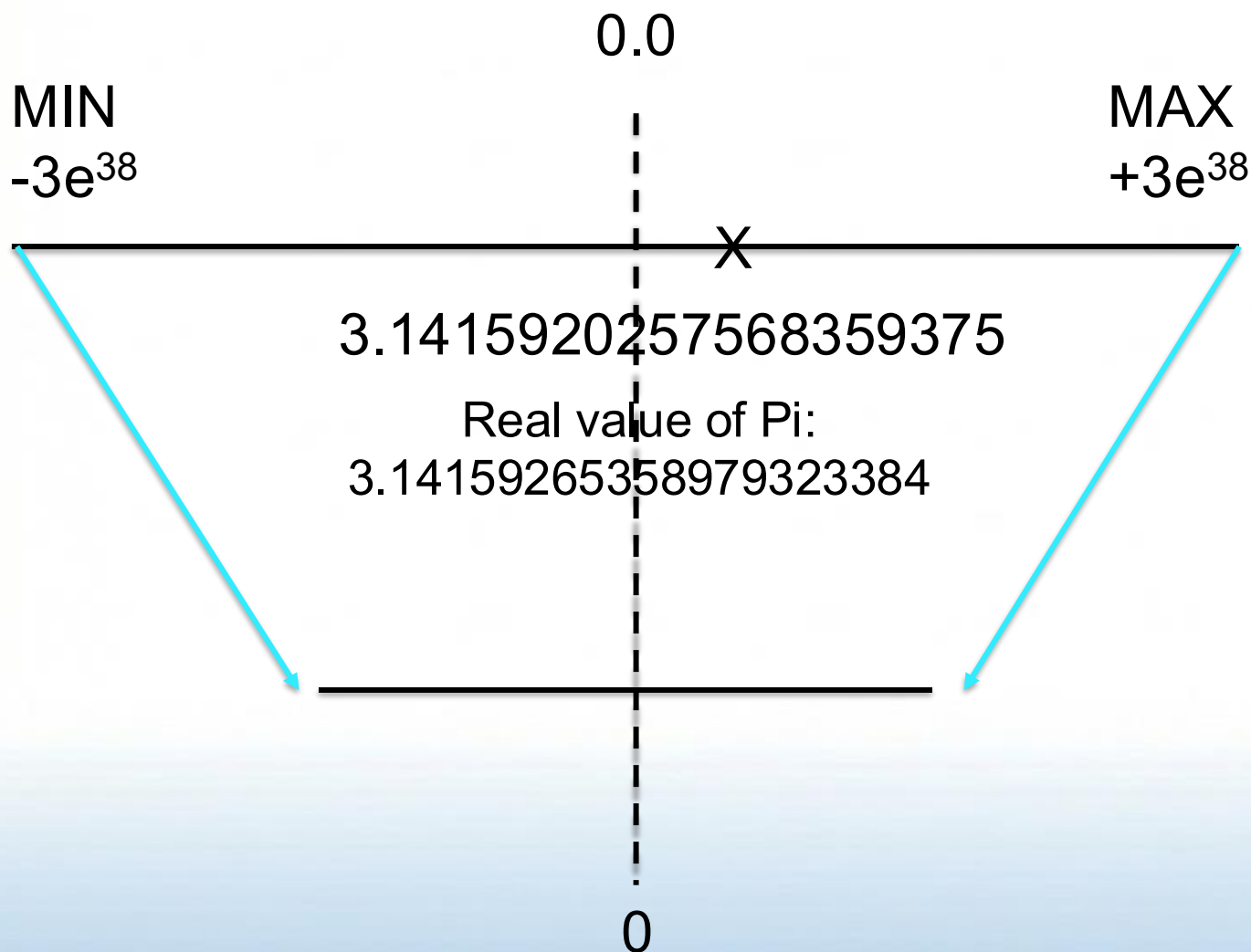




Quantization: FP32

Let's store Pi: 3.141592

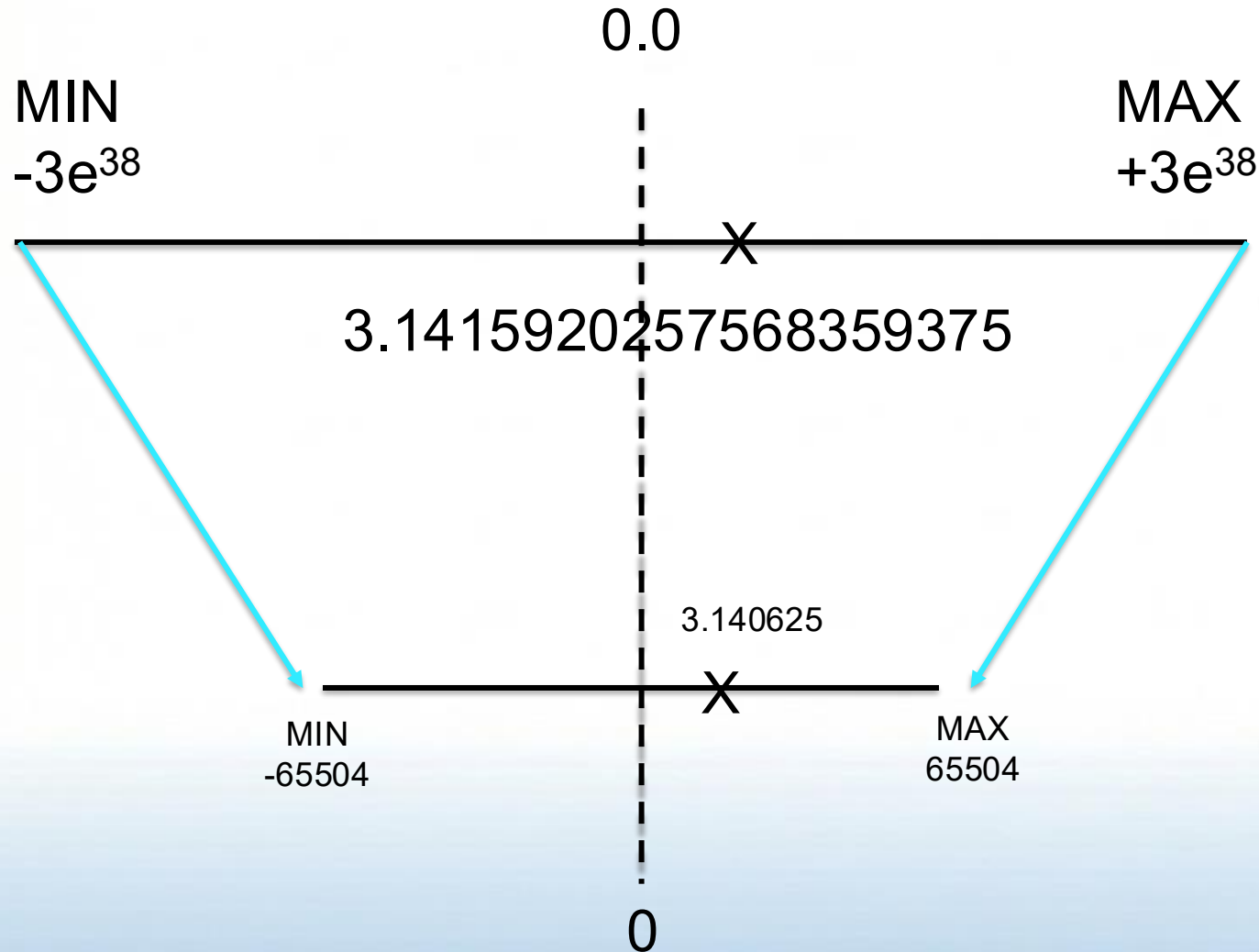
FP32 4 bytes memory



Mantissa/Significand
= Precision

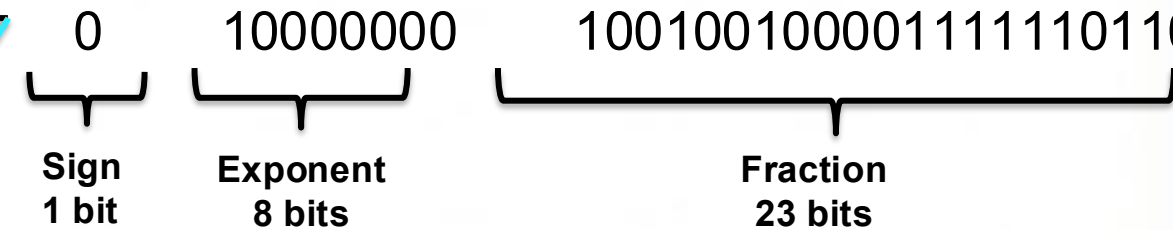


Quantization: FP16

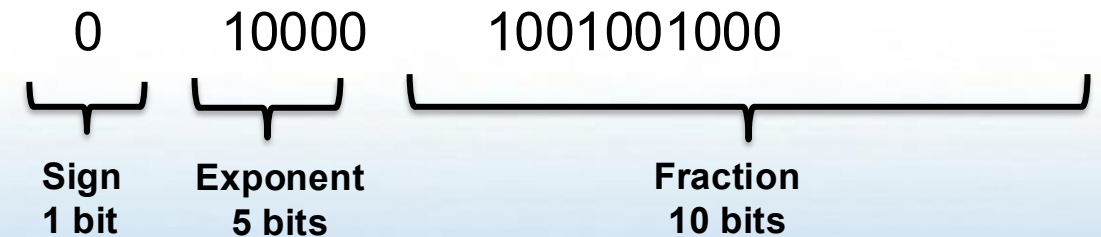


Let's store Pi: 3.141592

FP32 4 bytes memory

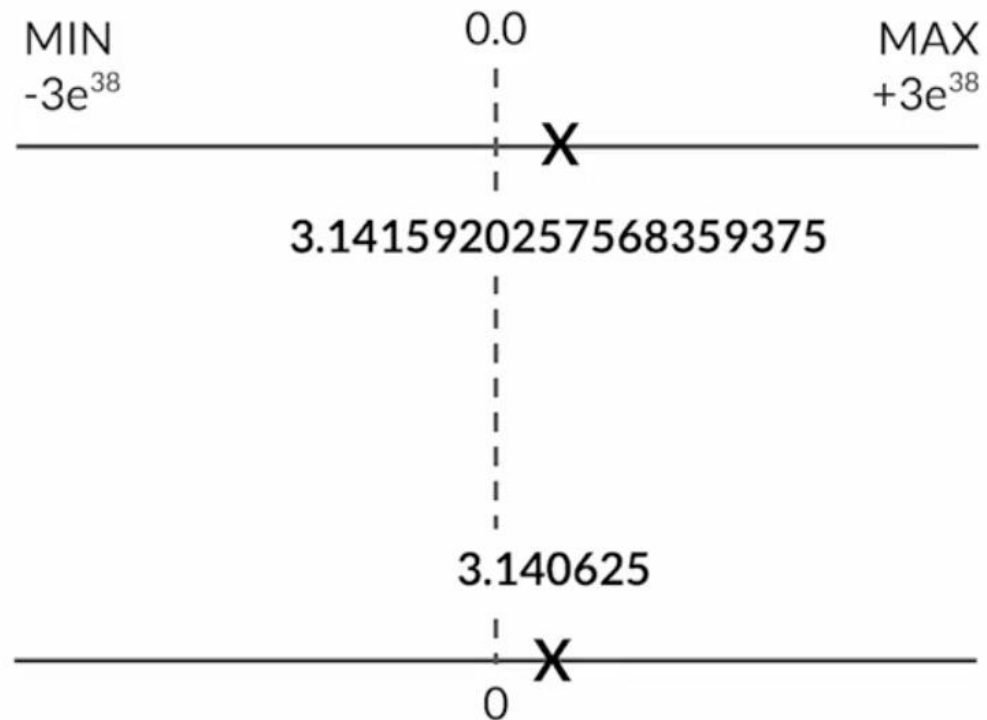


FP16 2 bytes memory





Quantization: BFLOAT16



Let's store Pi: 3.141592

FP32 4 bytes memory

0 10000000 10010010000111111011000

Sign Exponent Fraction

1 bit 8 bits 23 bits

BFLOAT16 | BF16

2 bytes memory

0 10000000 1001001

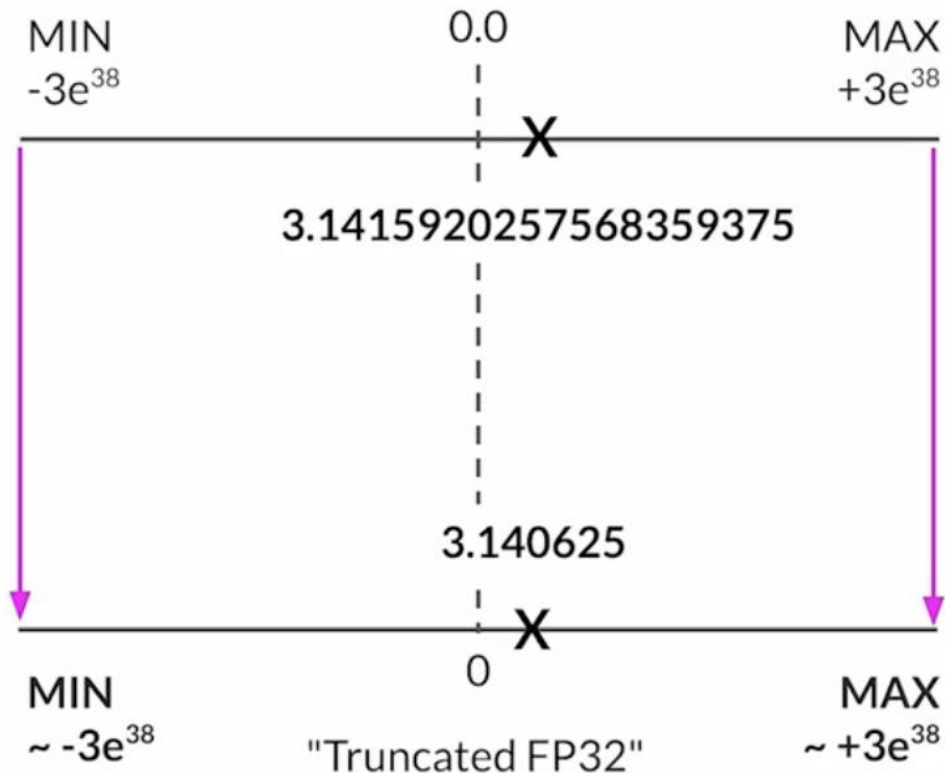
Sign Exponent Fraction

1 bit 8 bits 7 bits



Quantization: BFLOAT16

Let's store Pi: 3.141592



FP32 4 bytes memory

0 10000000 10010010000111111011000

Sign 1 bit Exponent 8 bits Fraction 23 bits

BFLOAT16 | BF16

2 bytes memory

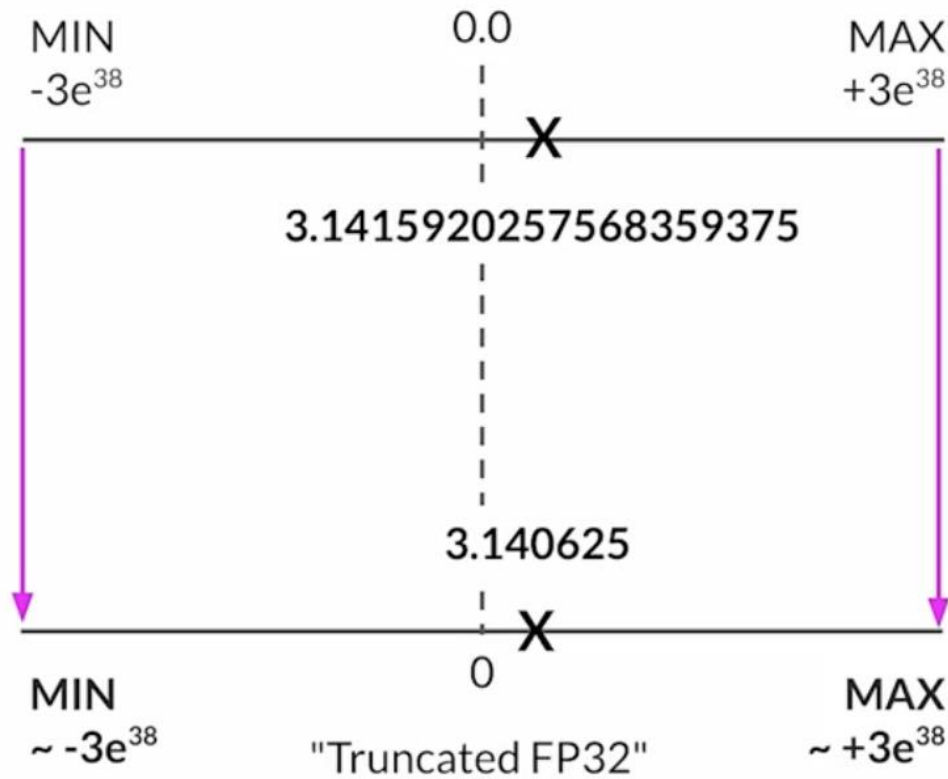
0 10000000 1001001

Sign 1 bit Exponent 8 bits Fraction 7 bits



Quantization: BFLOAT16

Let's store Pi: 3.141592



FP32 4 bytes memory

0 10000000 1001001000011111011000

Sign 1 bit Exponent 8 bits Fraction 23 bits

BFLOAT16 | BF16

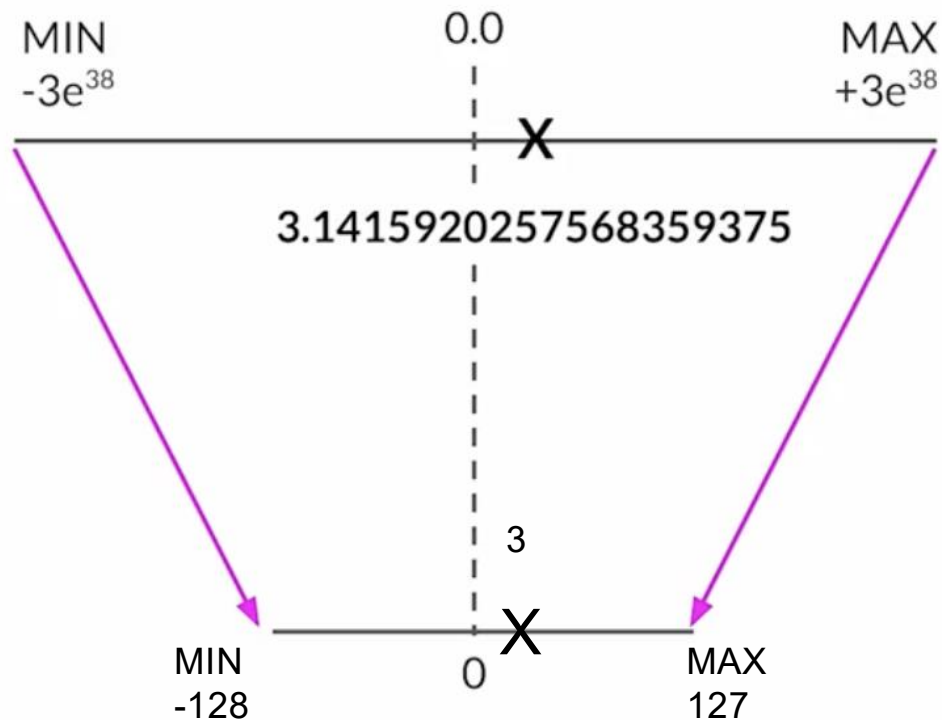
2 bytes memory

0 10000000 1001001

Sign 1 bit Exponent 8 bits Fraction 7 bits



Quantization: INT8



Let's store Pi: 3.141592

FP32 4 bytes memory

0	10000000	10010010000111111011000
Sign 1 bit	Exponent 8 bits	Fraction 23 bits

INT8 1 byte memory

0	0000011
Sign 1 bit	Fraction 7 bits



Quantization: Summary

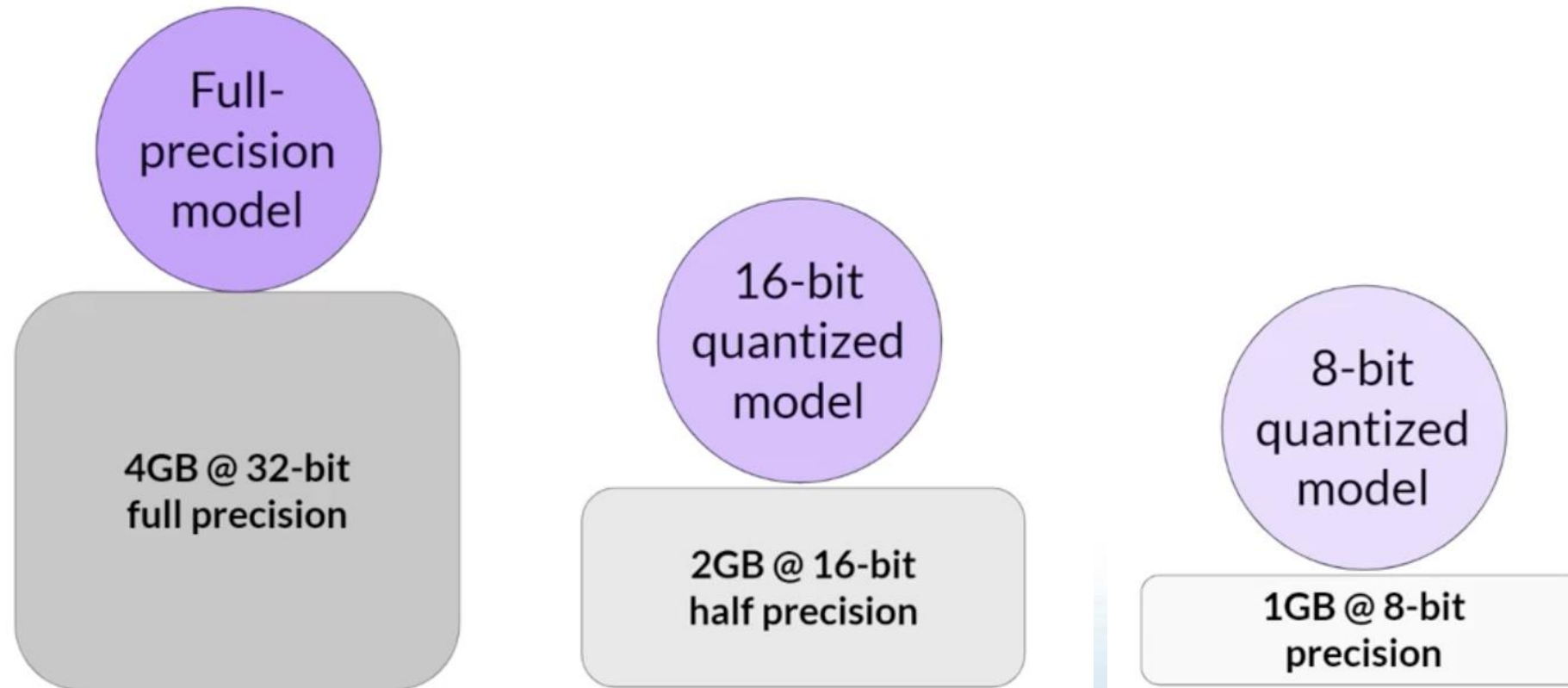
	Bits	Exponent	Fraction	Memory needed to store one value
FP32	32	8	23	4 bytes
FP16	16	5	10	2 bytes
BFLOAT16	16	8	7	2 bytes
INT8	8	-/-	7	1 byte



- Reduce required memory to store and train models.
- Statistically projects 32-bit floating point numbers into lower precision spaces .
- Quantization-aware training (QAT) learns the quantization scaling factors during training.
- BFLOAT16 is a popular choice.



Approximate GPU RAM needed to store 1B parameters





GPU RAM needed to train larger models

1B param
model

175B param
model

500B param
model

4,200 GB @ 32-bit
full precision

12,000 GB @ 32-bit
full precision





GPU RAM needed to train larger models

As model sizes get larger, you will need to split your model across multiple GPUs for training

**1B param
model**

4,200 GB @ 32-bit
full precision

**175B param
model**

**500B param
model**

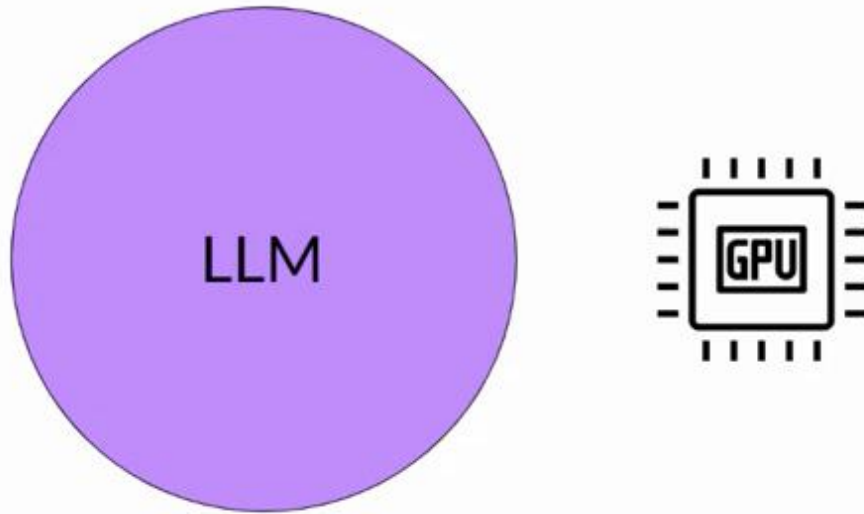
12,000 GB @ 32-bit
full precision

Efficient Multi-GPU Compute Strategies

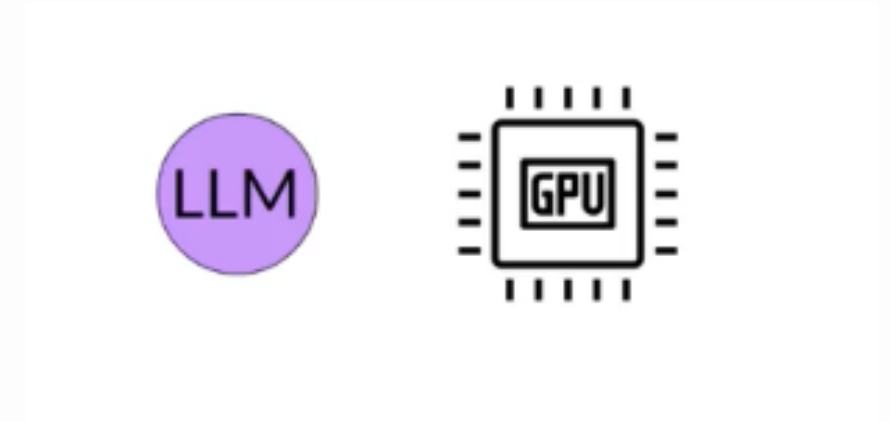




When to use distributed compute



Model too big for single GPU



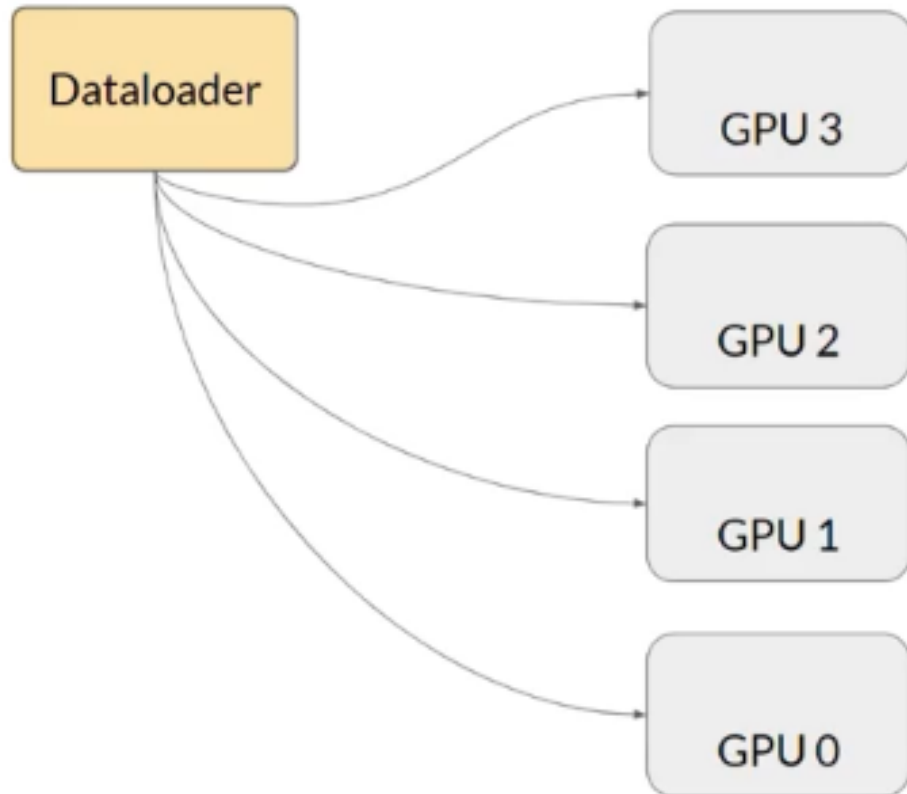


Distributed Data Parallel (DDP)

Dataloader

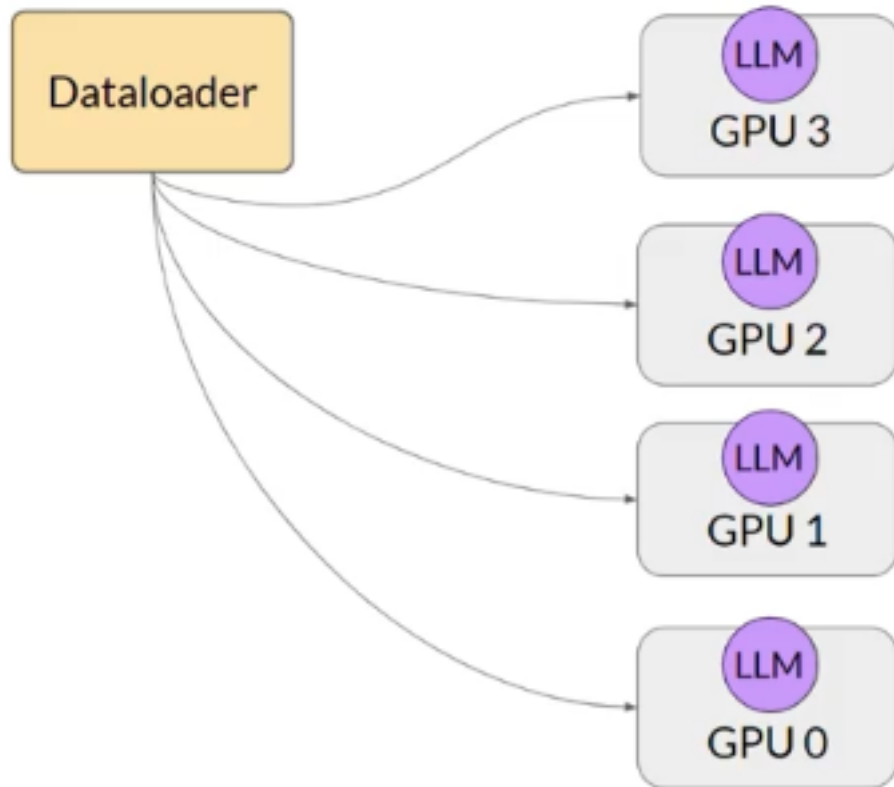


Distributed Data Parallel (DDP)



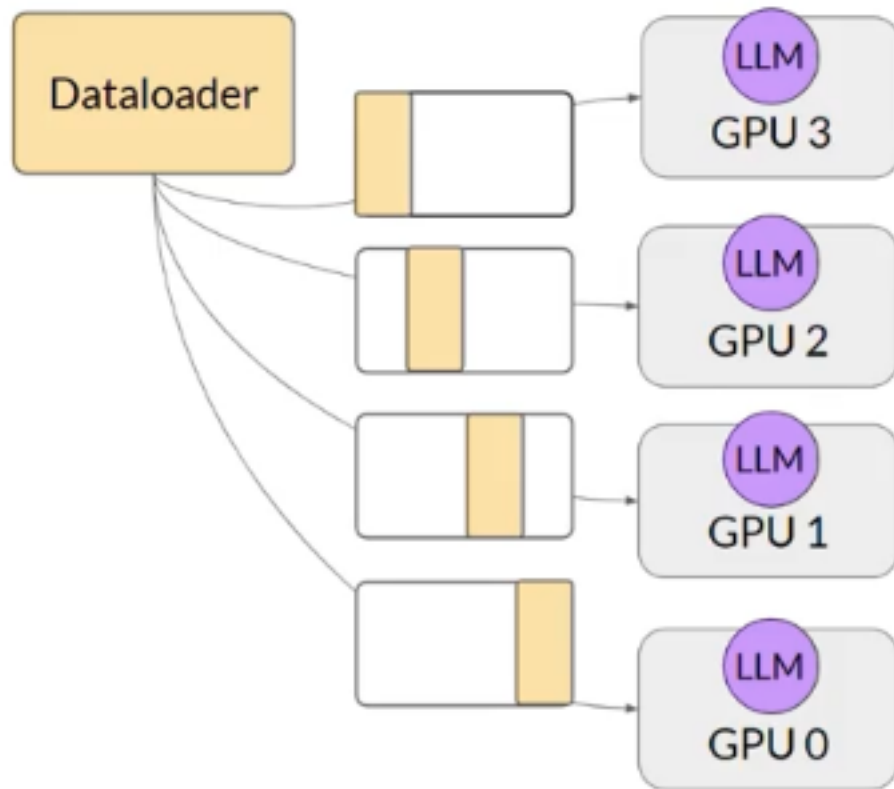


Distributed Data Parallel (DDP)



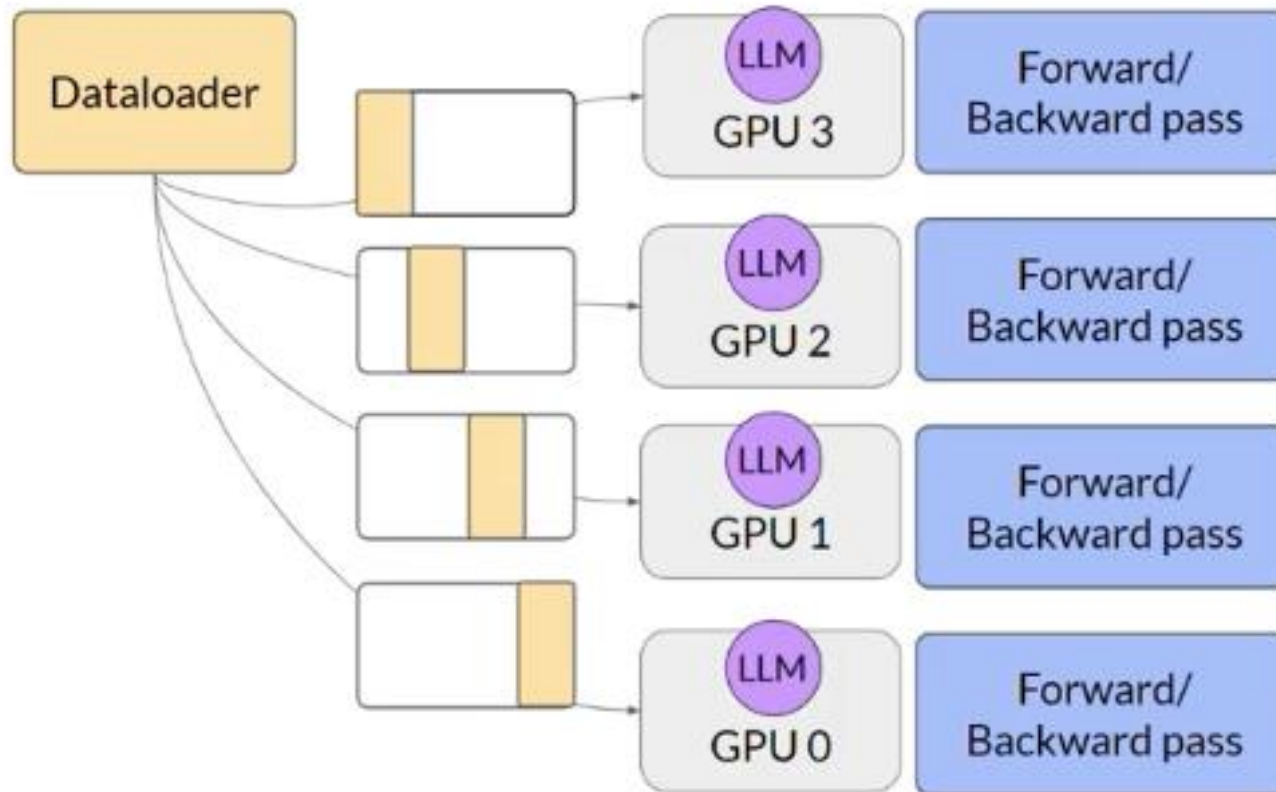


Distributed Data Parallel (DDP)



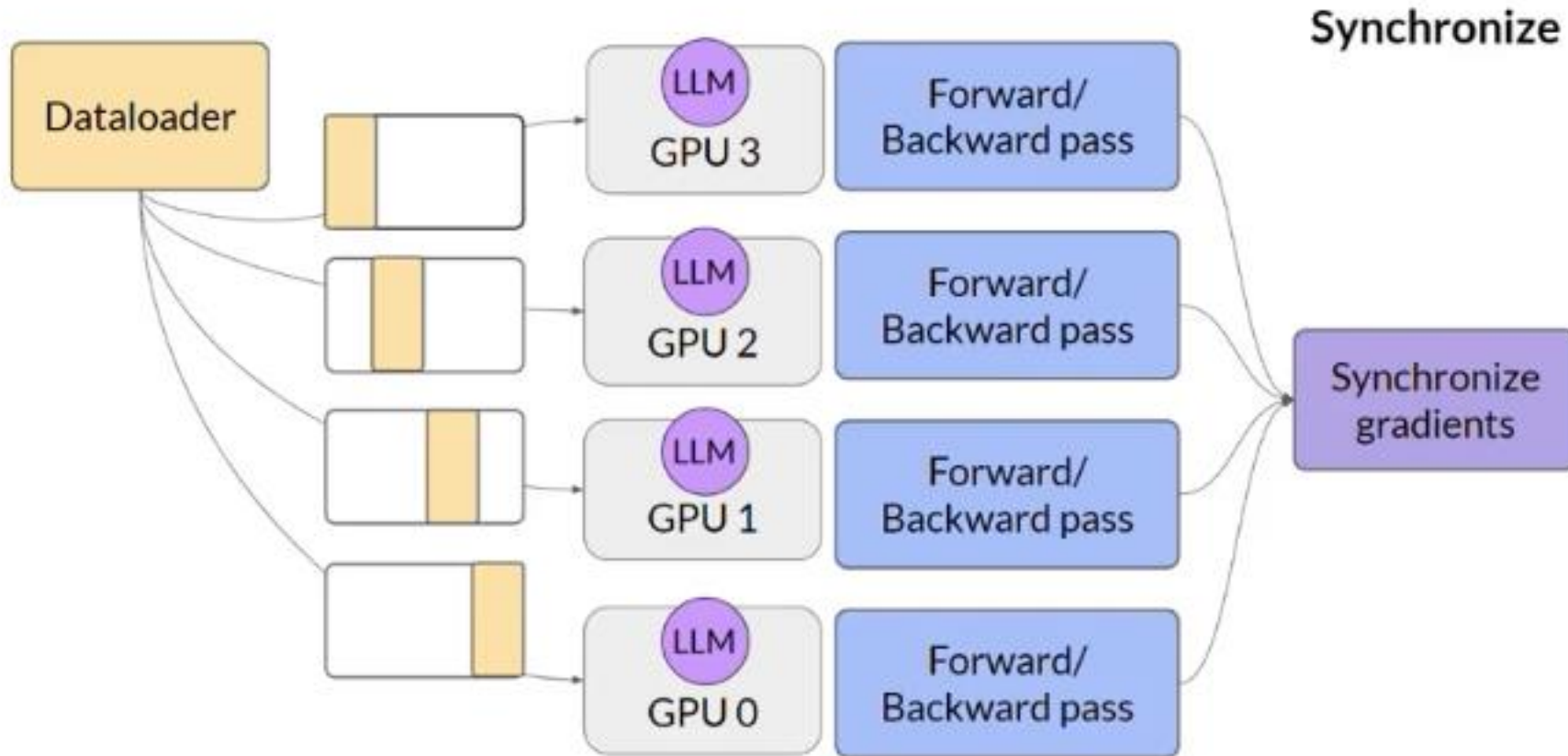


Distributed Data Parallel (DDP)



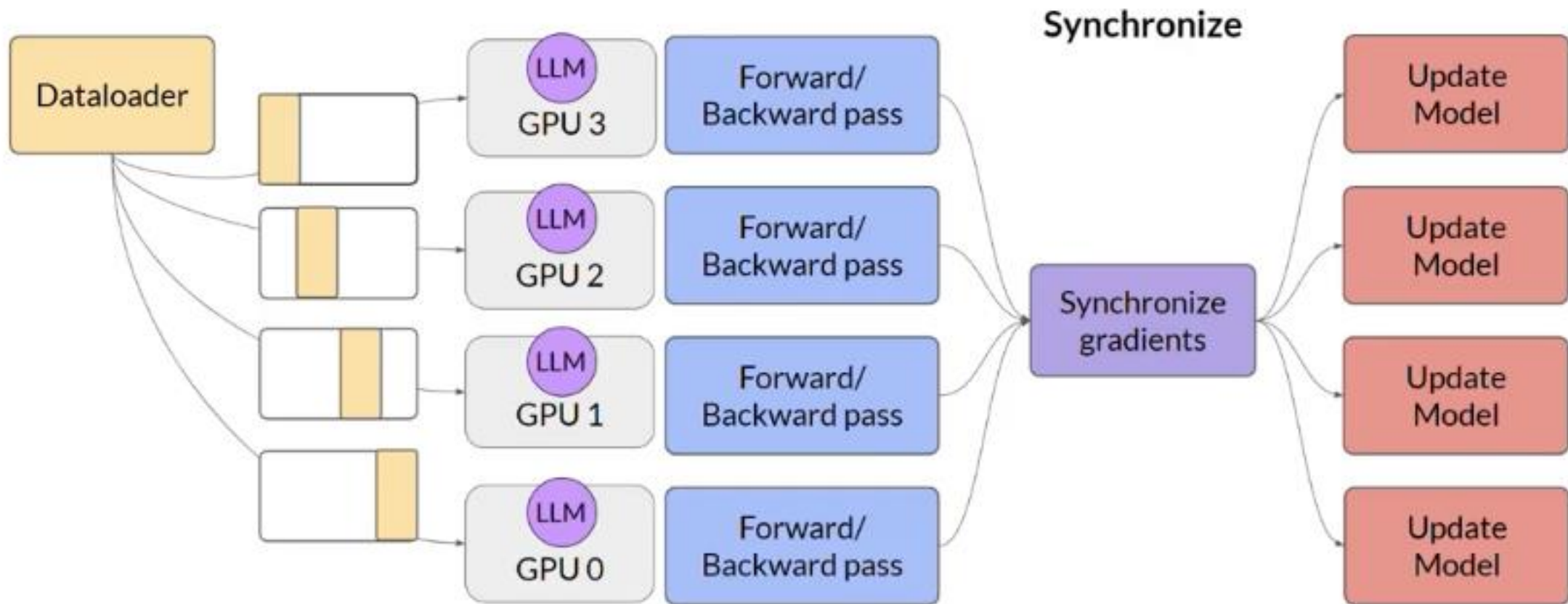


Distributed Data Parallel (DDP)





Distributed Data Parallel (DDP)





Fully Sharded Data Parallel (FSDP)

- Motivated by the “ZeRO” paper – zero data overlap between GPUs

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

Samyam Rajbhandari*, Jeff Rasley*, Olatunji Ruwase, Yuxiong He
{samyamr, jerasley, olruwase, yuxhe}@microsoft.com

ABSTRACT

Large deep learning models offer significant accuracy gains, but training billions to trillions of parameters is challenging. Existing solutions such as data and model parallelisms exhibit fundamental limitations to fit these models into limited device memory, while obtaining computation, communication and development efficiency. We develop a novel solution, Zero Redundancy Optimizer (*ZeRO*), to optimize memory, vastly improving training speed while increasing the model size that

common settings like mixed precision and ADAM optimizer [6]. Other existing solutions such as Pipeline Parallelism (PP), Model Parallelism (MP), CPU-Offloading, etc, make trade-offs between functionality, usability, as well as memory and compute/communication efficiency, all of which are crucial to training with speed and scale.

Among different existing solution for training large models, MP is perhaps the most promising one. The largest models in the current literature, the 11B T5 model [5], and Megatron-

S. Rajbhandari, J. Rasley, O. Ruwase and Y. He, "ZeRO: Memory optimizations Toward Training Trillion Parameter Models," SC20: *International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2020, pp. 1-16, doi: 10.1109/SC41405.2020.00024.



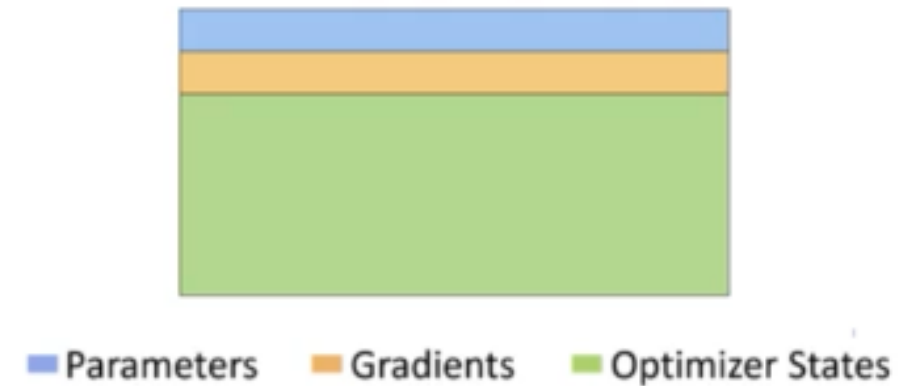
Recap: Additional GPU RAM needed for training

	Bytes per parameter
Model Parameters (Weights)	4 bytes per parameter
Adam optimizer (2 states)	+8 bytes per parameter
Gradients	+4 bytes per parameter
Activations and temp memory (variable size)	+8 bytes per parameter (high-end estimate)
TOTAL	=4 bytes per parameter +20 extra bytes per parameter



Memory usage in DDP

- One full copy of model and training parameters on each GPU



Sources:

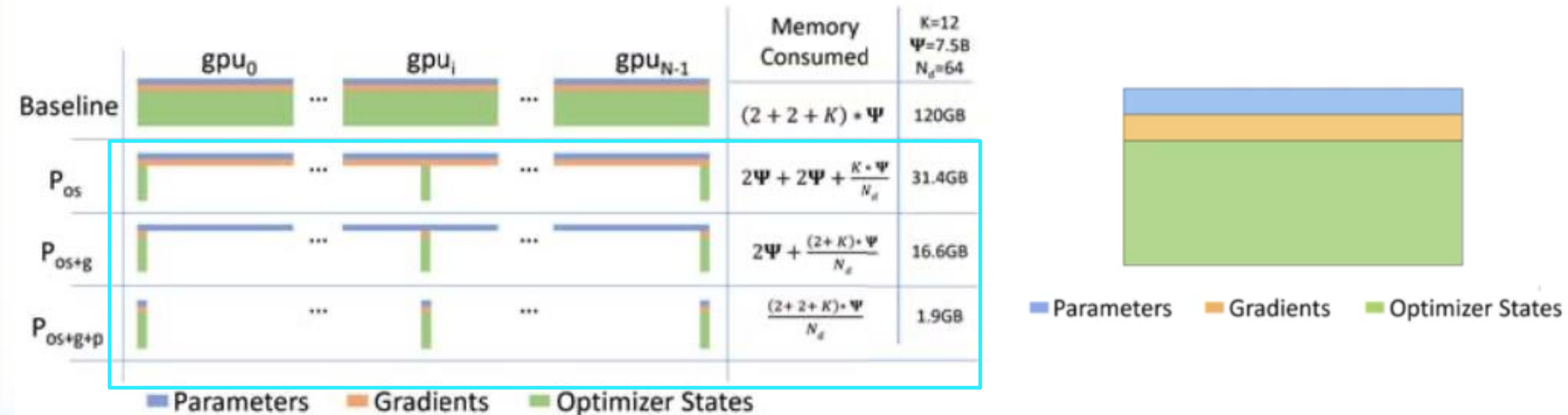
Rajbhandari et al. 2019: "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"

Zhao et al. 2023: "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel"



Zero Redundancy Optimizer (ZeRO)

- Reduces memory by distributing (sharding) the model



Sources:

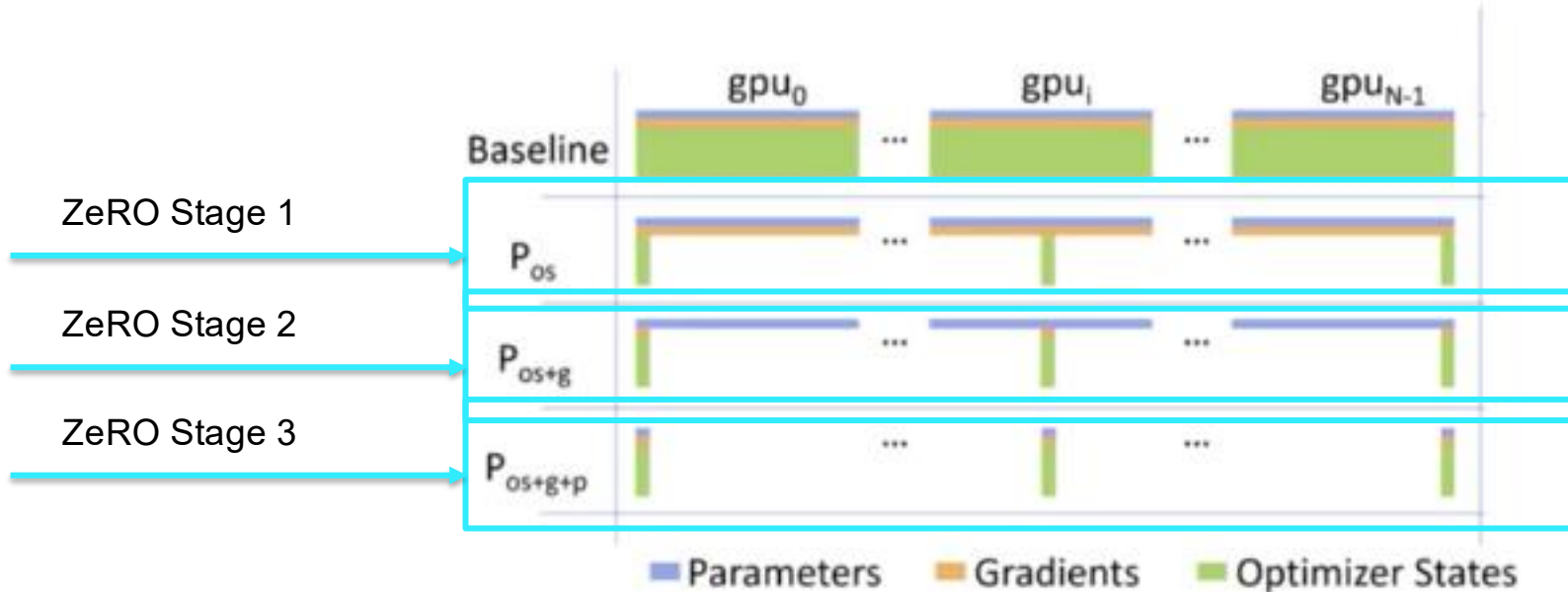
Rajbhandari et al. 2019: "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"

Zhao et al. 2023: "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel"



Zero Redundancy Optimizer (ZeRO)

- Reduces memory by distributing (sharding) the model parameters, gradients



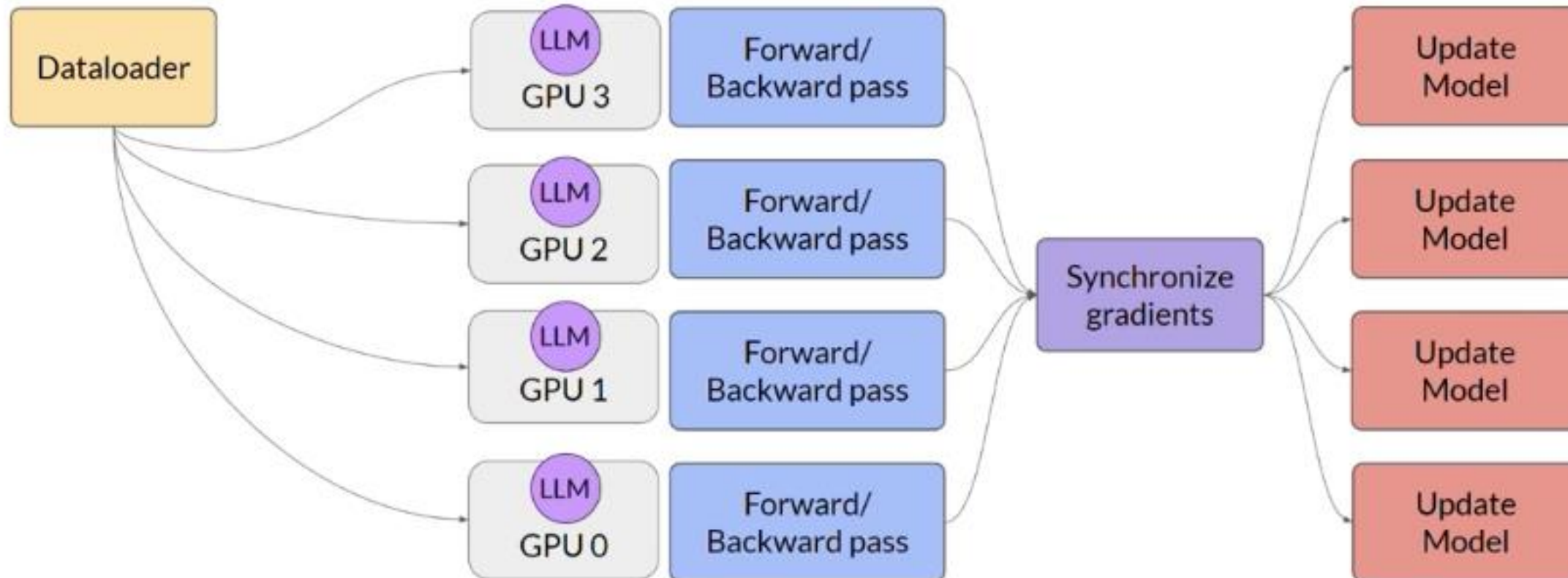
Sources:

Rajbhandari et al. 2019: "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"

Zhao et al. 2023: "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel"

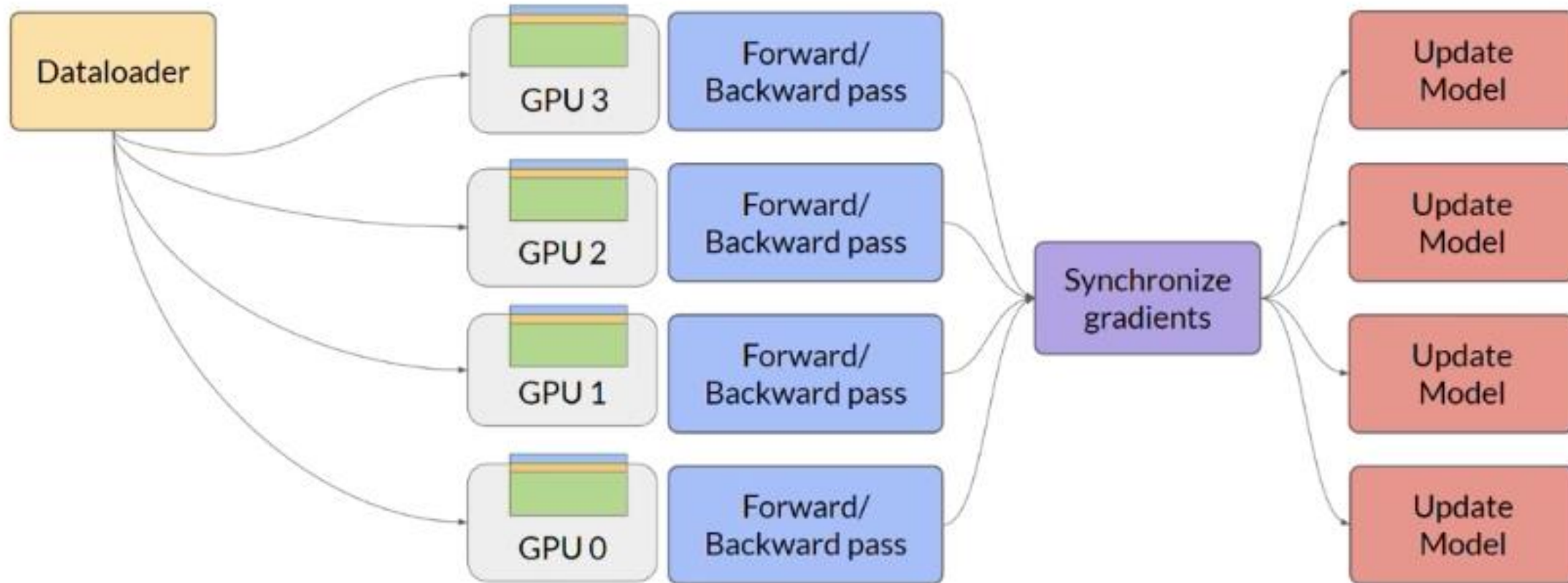


Distributed Data Parallel (DDP)



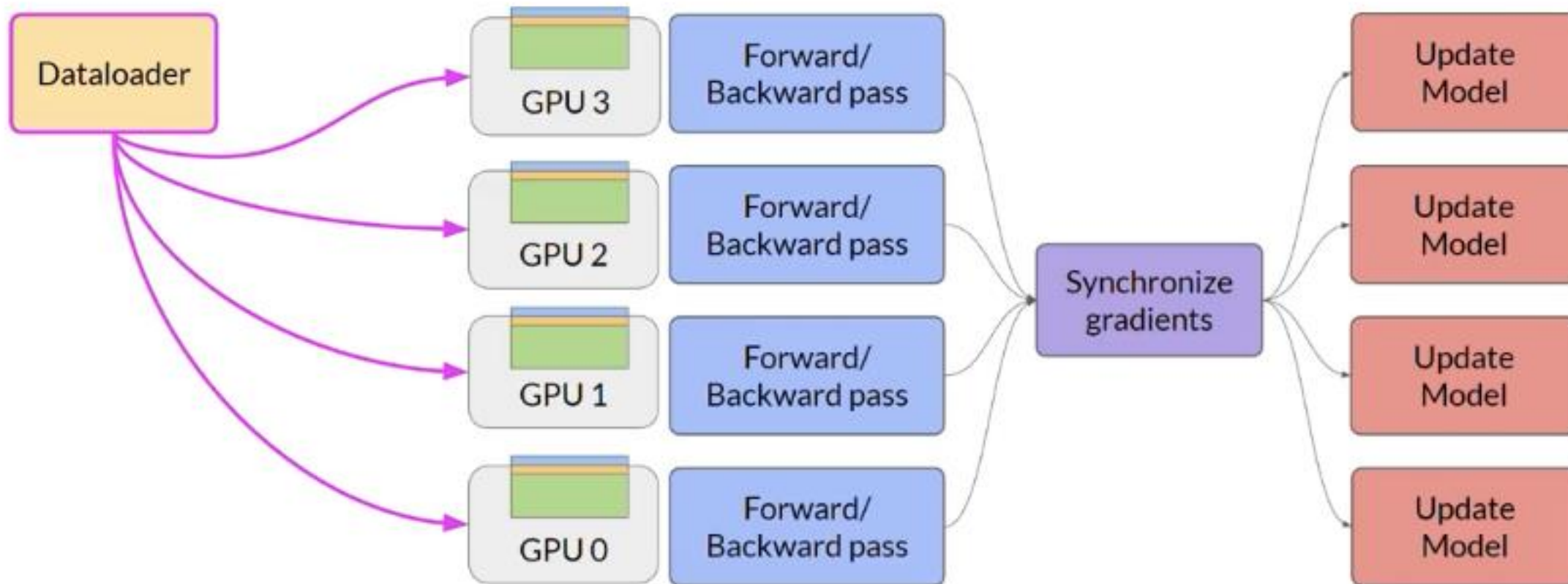


Distributed Data Parallel (DDP)



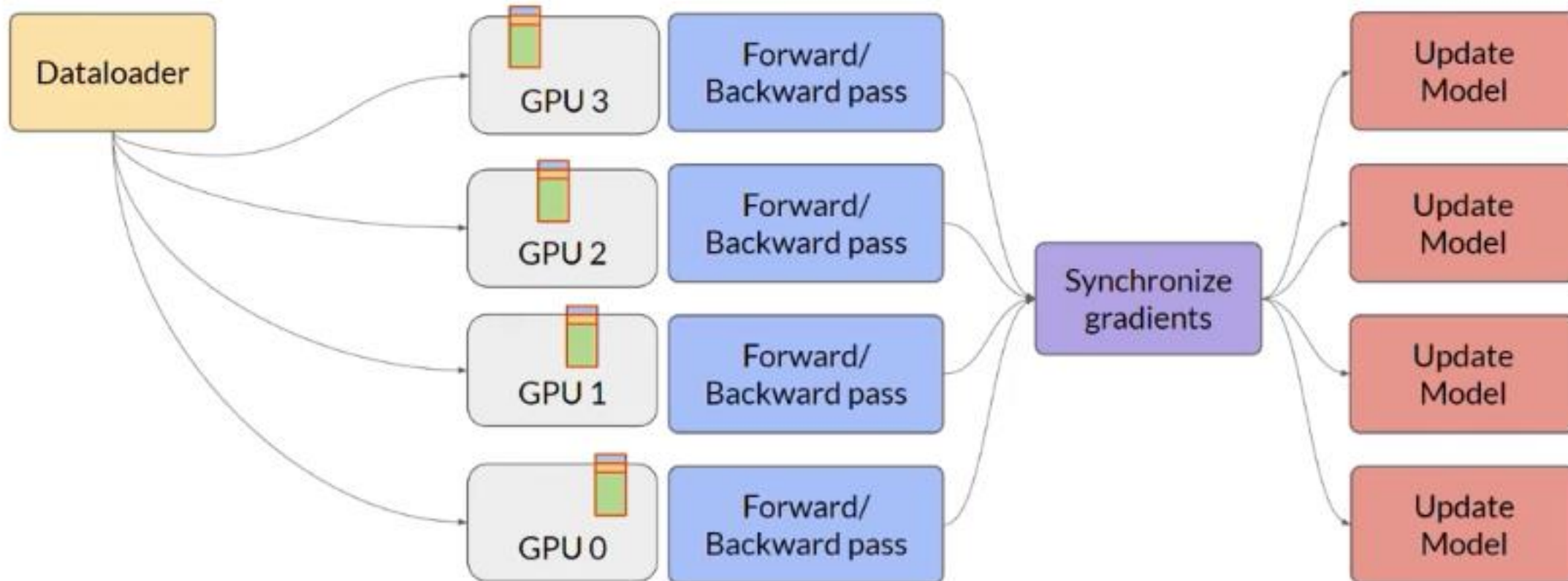


Fully Sharded Data Parallel (FSDP)



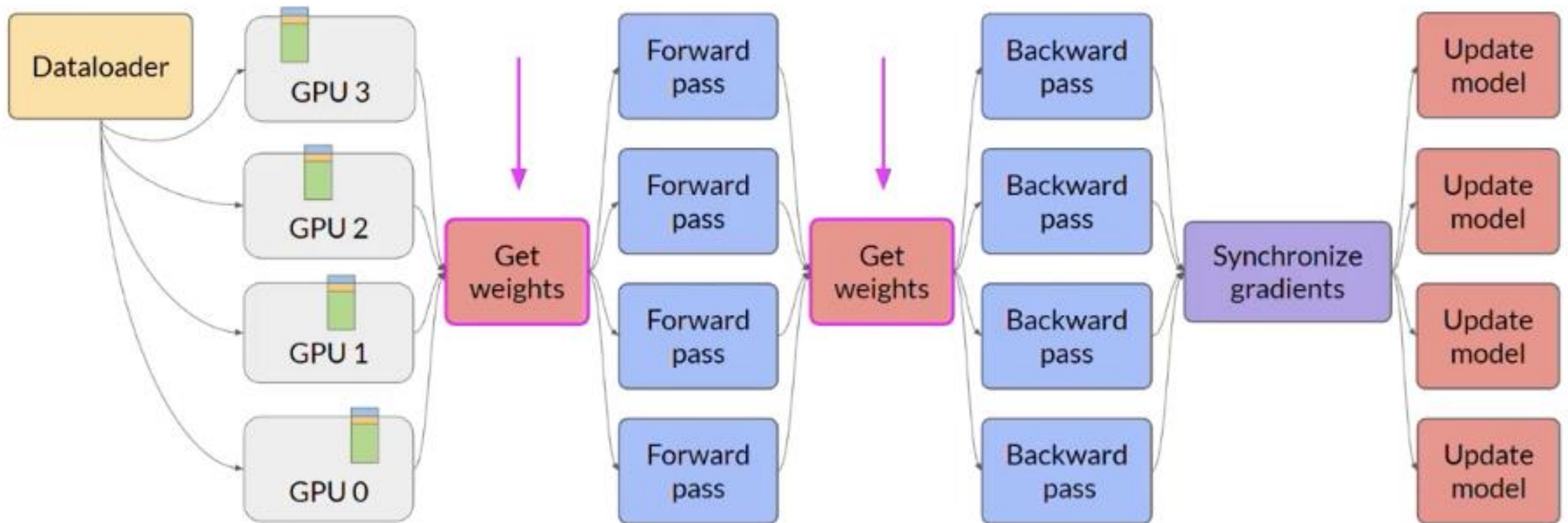


Fully Sharded Data Parallel (FSDP)



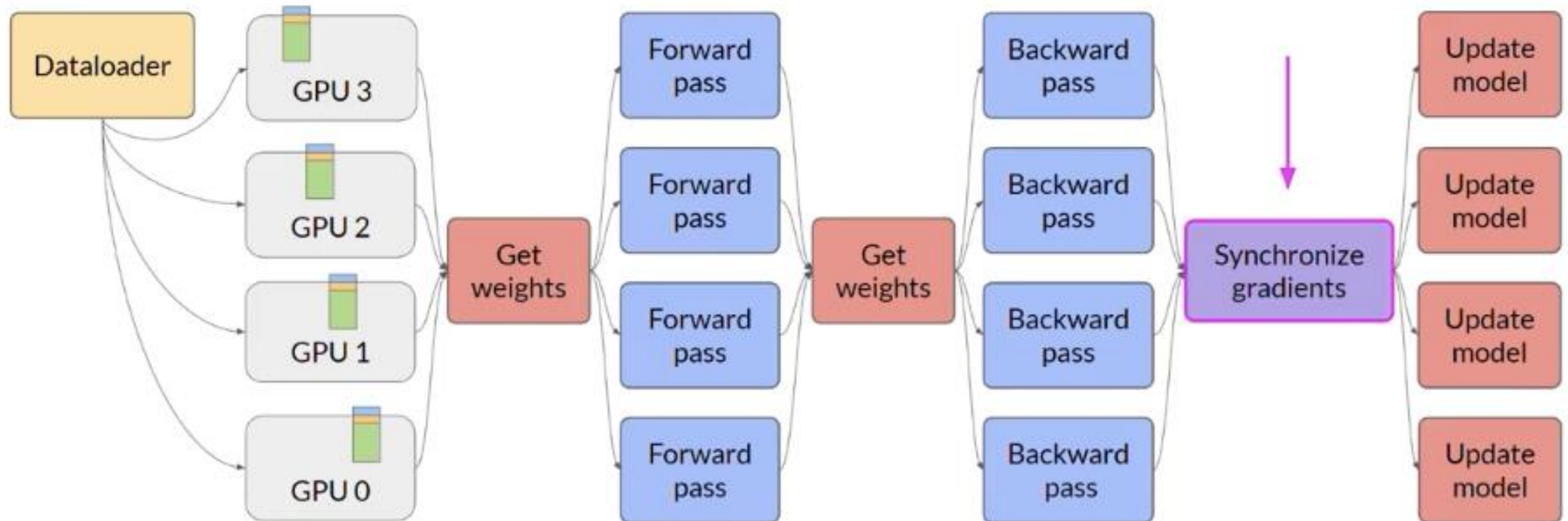


Fully Sharded Data Parallel (FSDP)





Fully Sharded Data Parallel (FSDP)





Fully Sharded Data Parallel (FSDP)

- Helps to reduce overall GPU memory utilization
- Supports offloading to CPU if needed
- Configure level of sharding via sharding factor

Full replication (no sharding)

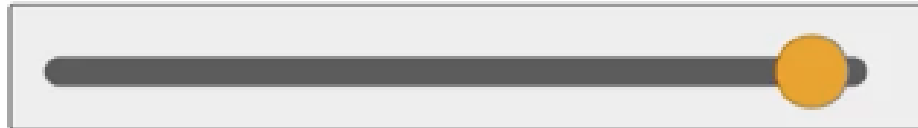
1 GPU



max. number of GPUs

Full sharding

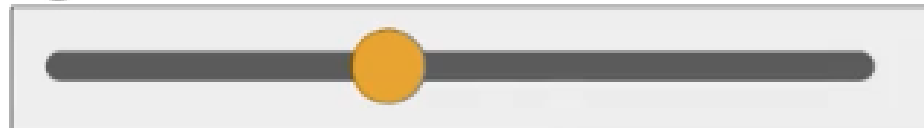
1 GPU



max. number of GPUs

Hybrid sharding

1 GPU

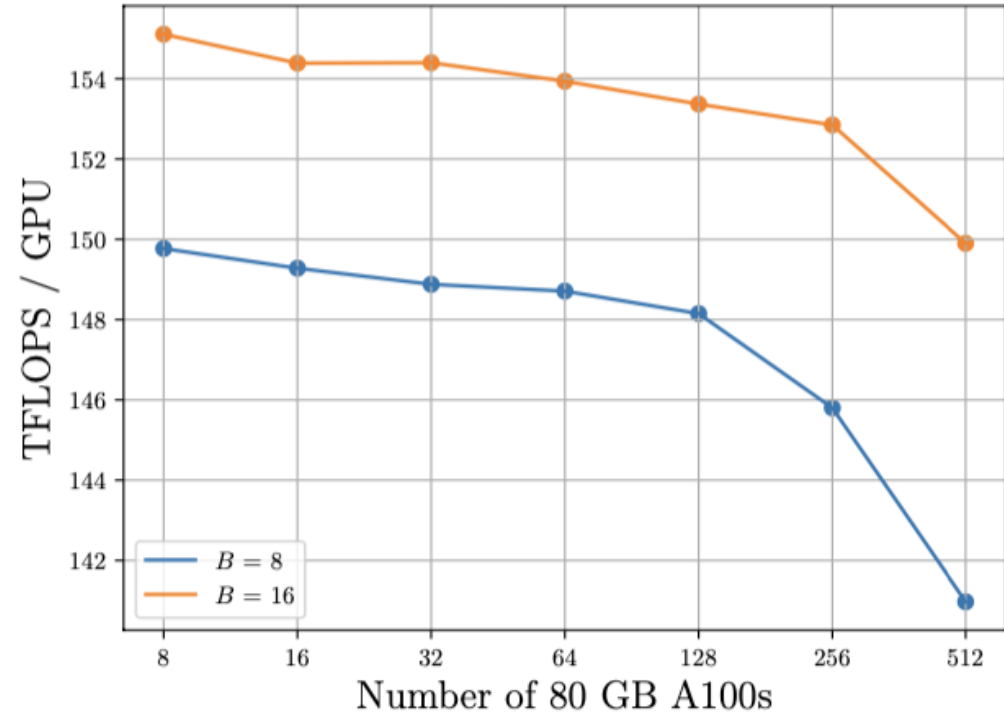
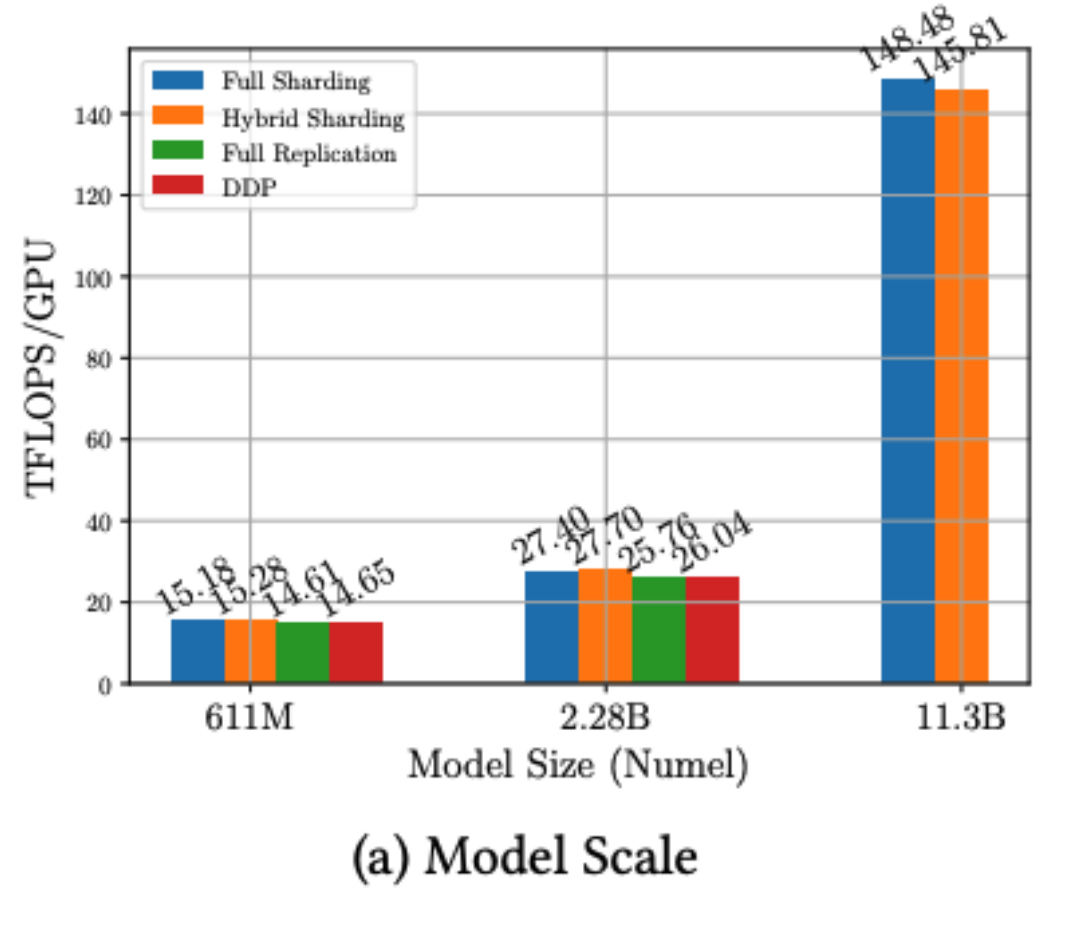


max. number of GPUs



Impact of using FSDP

Note: 1 teraflop/s = 1,000,000,000,000
(One trillion) floating point operations per second



(c) T5-11B TFLOPS



Scaling laws and compute-optimal models



Scaling choices for pre-training

Goal: maximize model
performance

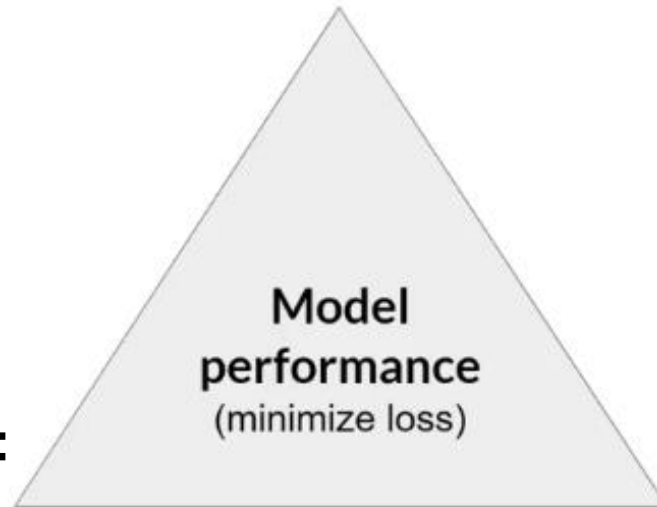
CONSTRAINT:

Compute budget
(GPUs, training time, cost)



SCALING CHOICE:

Dataset size
(number of tokens)



Model
performance
(minimize loss)

SCALING CHOICE:

Model size
(number of parameters)



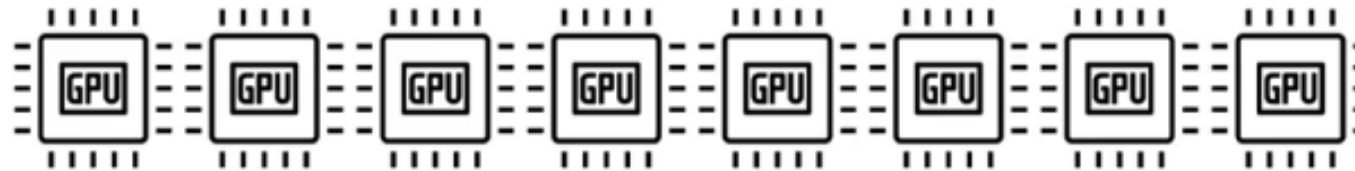


Compute budget for training LLMs

1 “petaflop/s-day” =

floating point operations performed at rate of 1 petaFLOP per second for one day

NVIDIA V100s

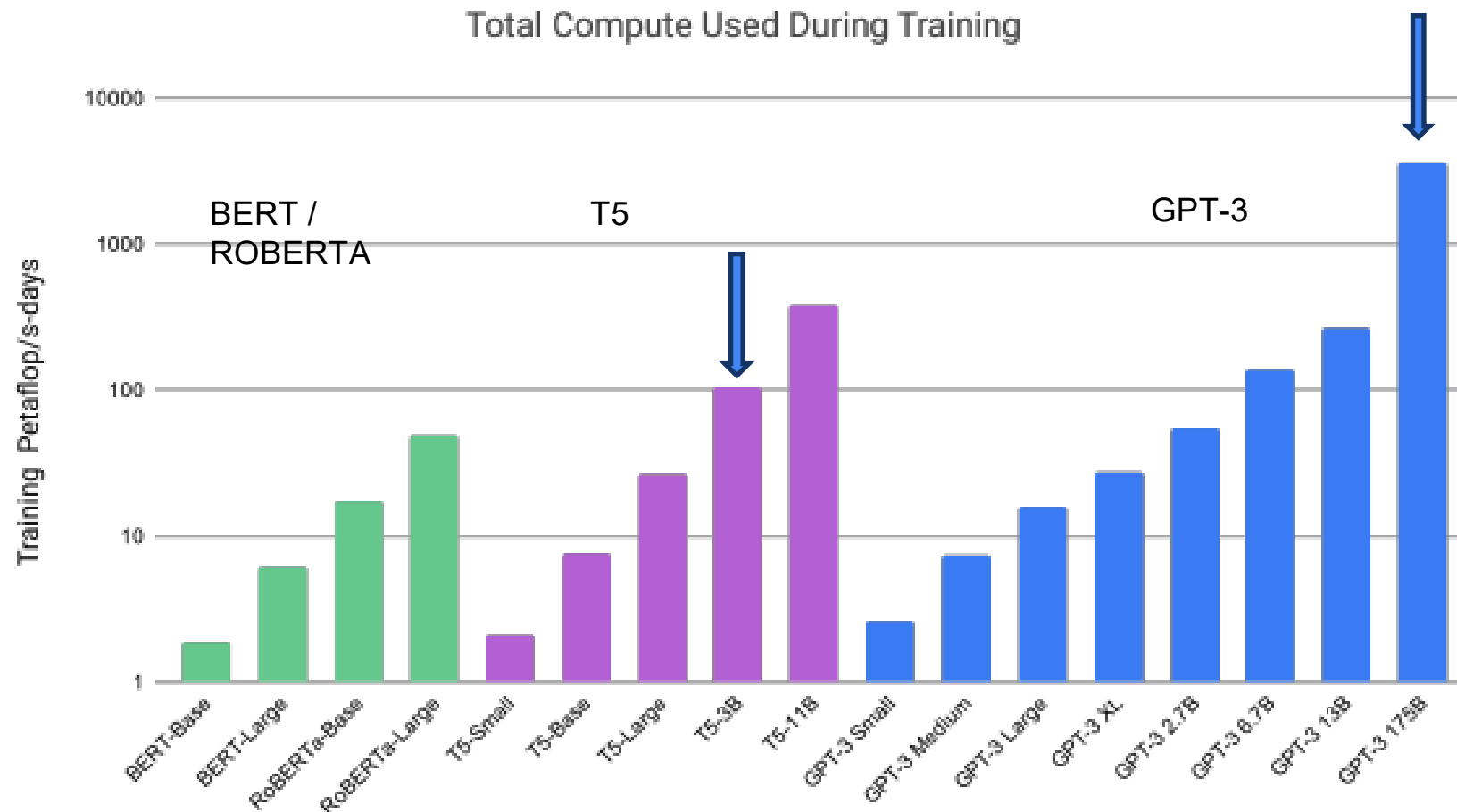


**Note: 1 petaFLOP/s = 1,000,000,000,000,000
(one quadrillion) floating point operations per second**

1 petaflops/s-day is these chips running at full efficiency for 24 hours



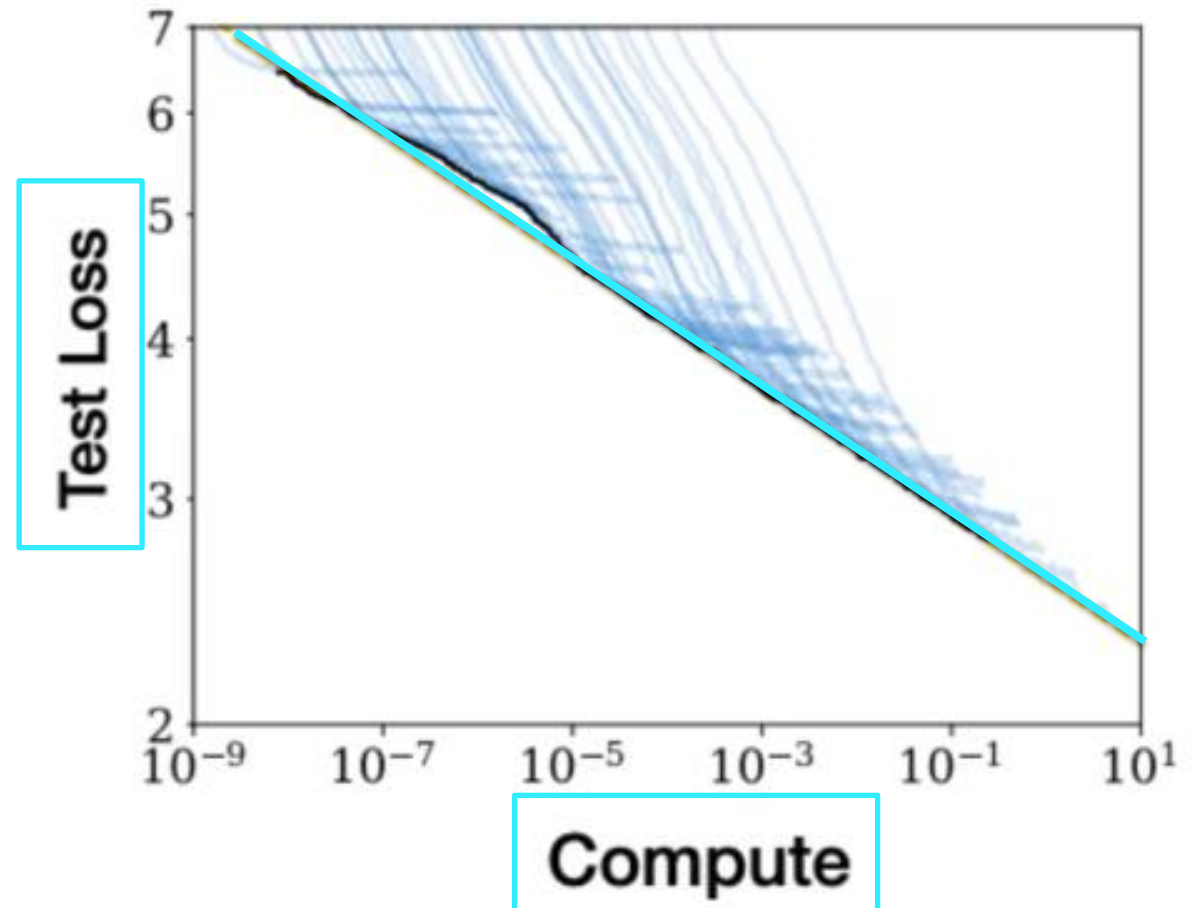
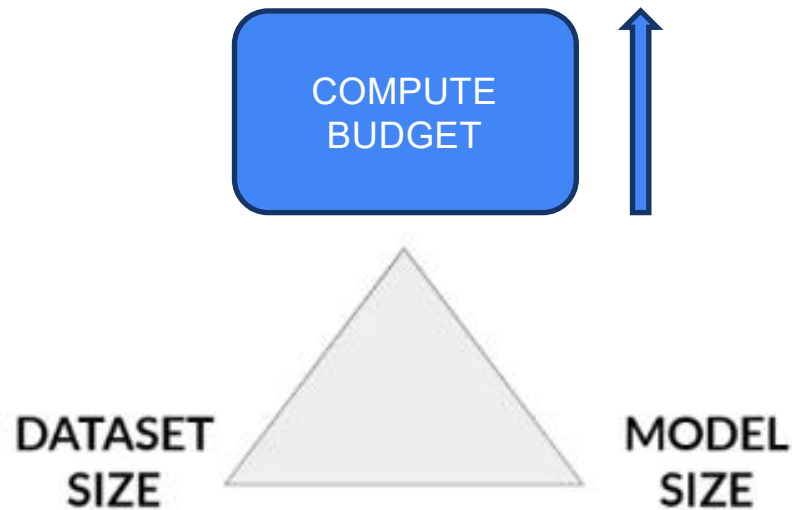
Number of petaflop/s-days to pretrain various LLMs



Brown et al. 2020, "Language Models are Few-Shot Learners", <https://arxiv.org/pdf/2005.14165>



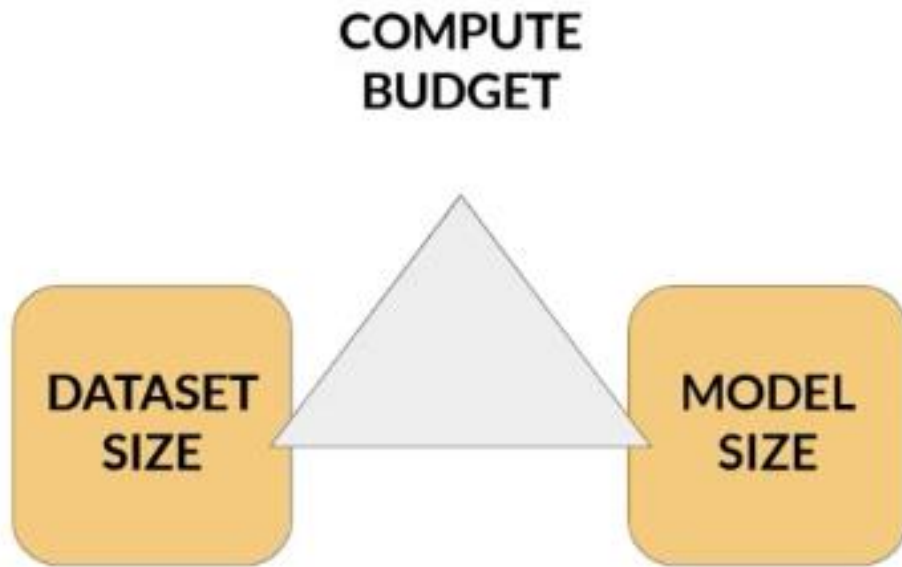
Compute budget vs. model performance



Source: Kaplan et al. 2020, "Scaling Laws for Neural Language Models"



Dataset size and model size vs.



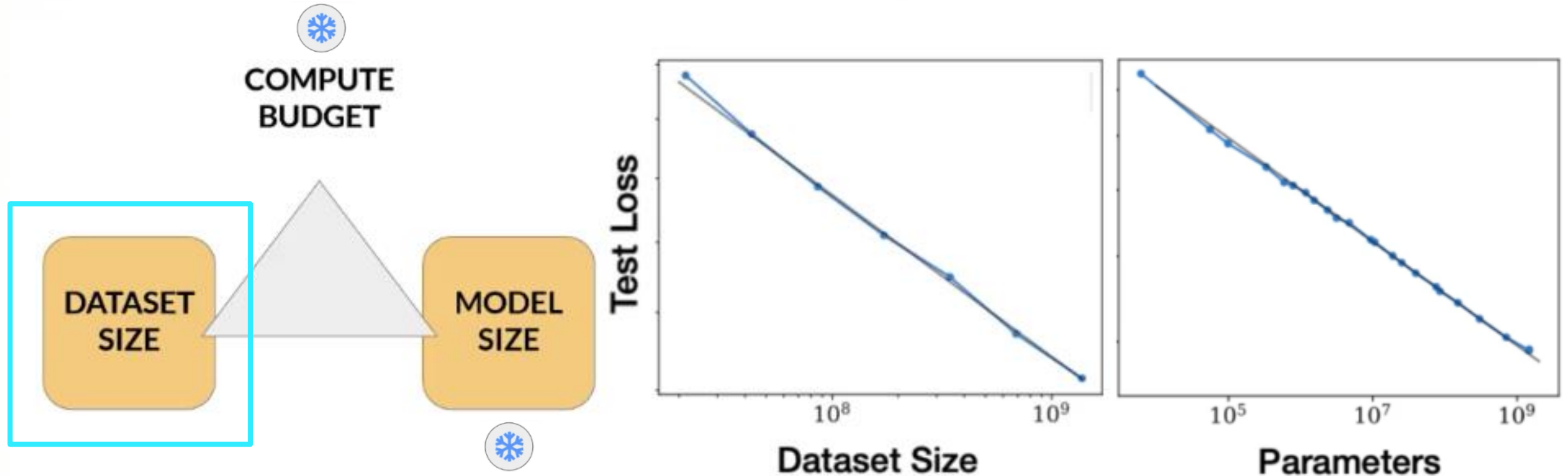
Compute resource constraints

- Hardware
- Project timeline
- Financial budget

Source: Kaplan et al. 2020, "Scaling Laws for Neural Language Models"



Data size and model size vs. performance



Source: Kaplan et al. 2020, "Scaling Laws for Neural Language Models"



Chinchilla paper



Training Compute-Optimal Large Language Models

Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre*

*Equal contributions

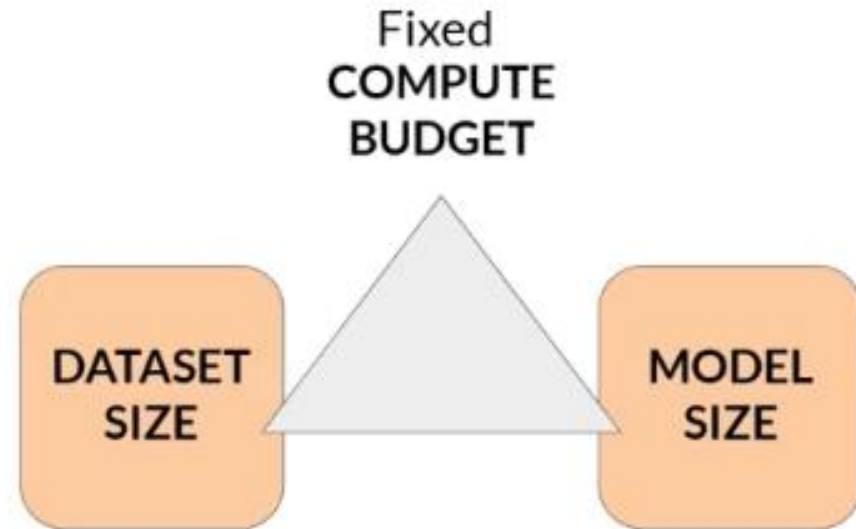
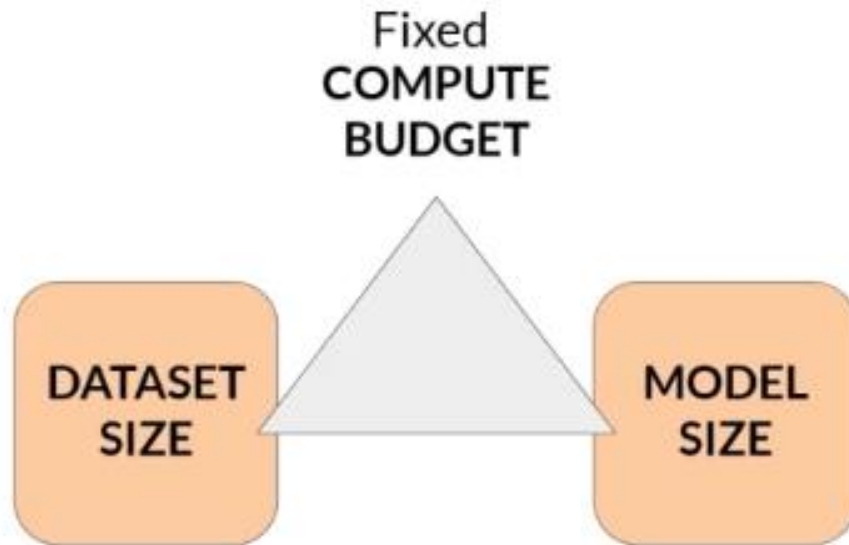
We investigate the optimal model size and number of tokens for training a transformer language model under a given compute budget. We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, we find that for compute-optimal training, the model size and the number of training tokens should be scaled equally: for every doubling of model size the number of training tokens should also be doubled. We test this hypothesis by training a predicted compute-optimal model, **Chinchilla**, that uses the same compute budget as *Gopher* but with 70B parameters and 4× more data. *Chinchilla* uniformly and significantly outperforms *Gopher* (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B) on a large range of downstream evaluation tasks. This also means that *Chinchilla* uses substantially less compute for fine-tuning and inference, greatly facilitating downstream usage. As a highlight, *Chinchilla* reaches a state-of-the-art average accuracy of 67.5% on the MMLU benchmark, greater than a 7% improvement over *Gopher*.

Jordan et al. 2022



Compute optimal models

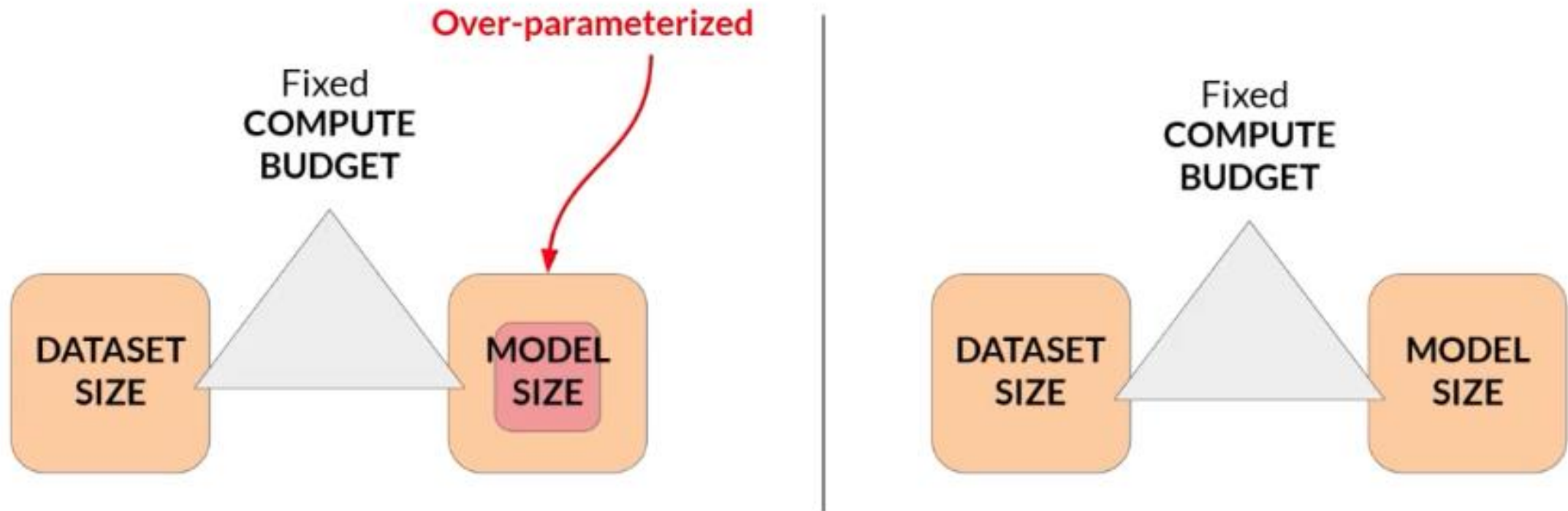
- Very large models may be **over-parameterized** and **under-trained**





Compute optimal models

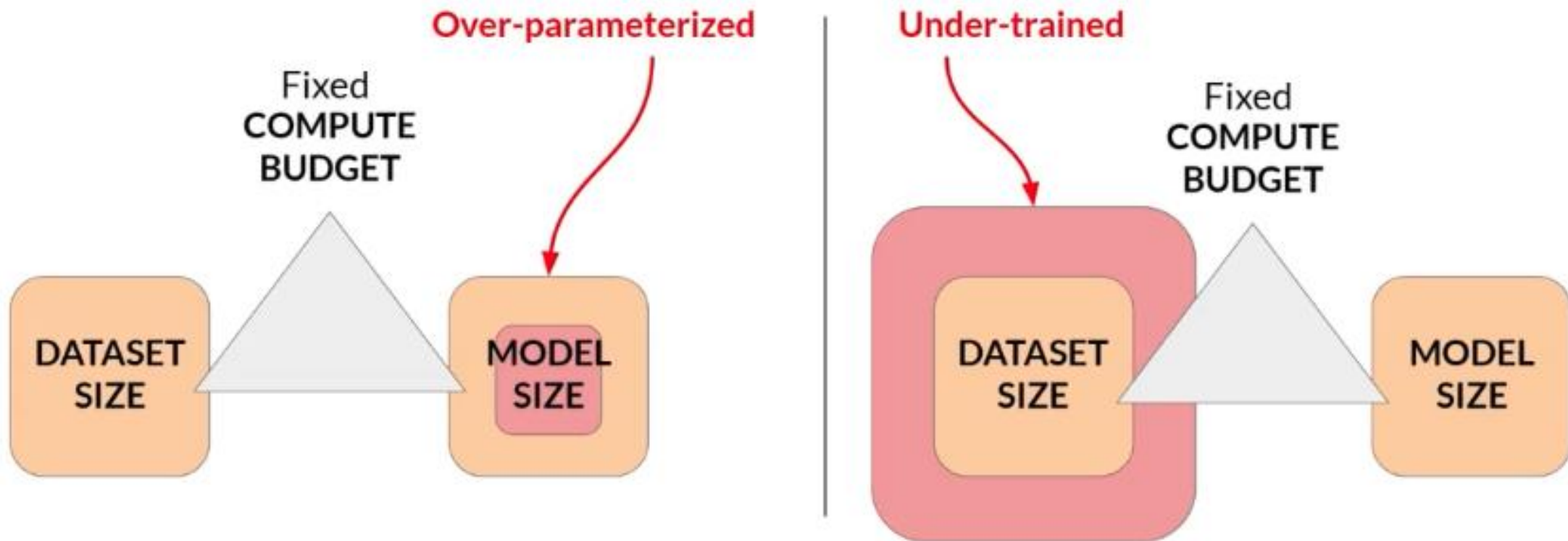
- Very large models may be **over-parameterized** and **under-trained**





Compute optimal models

- Very large models may be **over-parameterized** and **under-trained**
- Smaller models trained on more data could perform as well as large models





Chinchilla scaling laws for model and dataset size

Model	# of parameters	Compute-optimal* # of tokens (~20x)	Actual # tokens
Chinchilla	70B	~1.4T	1.4T
LLaMA-65B	65B	~1.3T	1.4T
GPT-3	175B	~3.5T	300B
OPT-175B	175B	~3.5T	180B
BLOOM	176B	~3.5T	350B

Compute optimal training datasize
is ~**20x** number of parameters

Sources: Hoffmann et al. 2022, "Training Compute-Optimal Large Language Models"
Touvron et al. 2023, "LLaMA: Open and Efficient Foundation Language Models"

* assuming models are trained to be
compute-optimal per Chinchilla paper



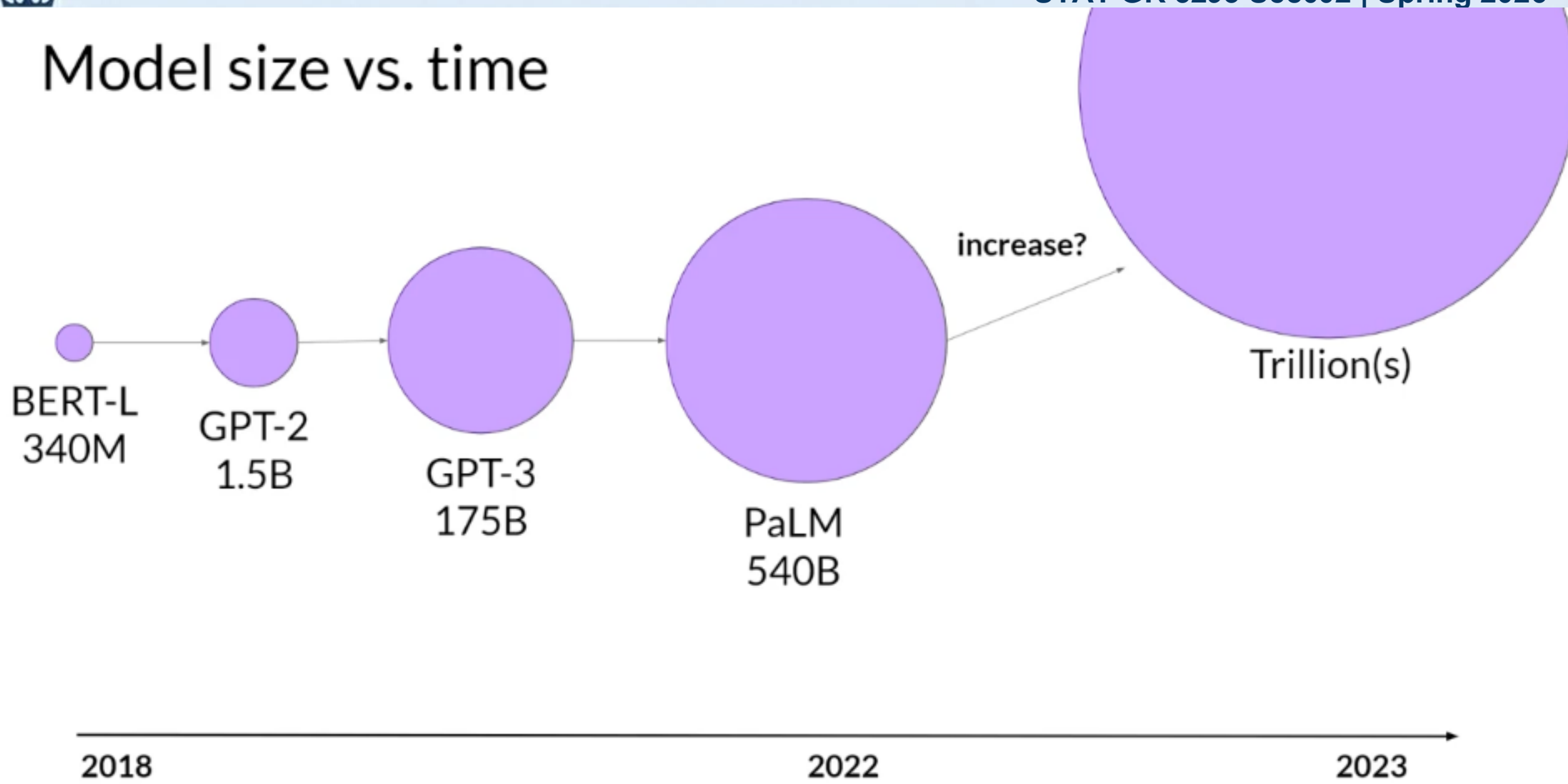
Question

Scaling laws for pre-training large language models consider several aspects to maximize performance of a model within a set of constraints and available scaling choices. Select all alternatives that should be considered for scaling when performing model pre-training?

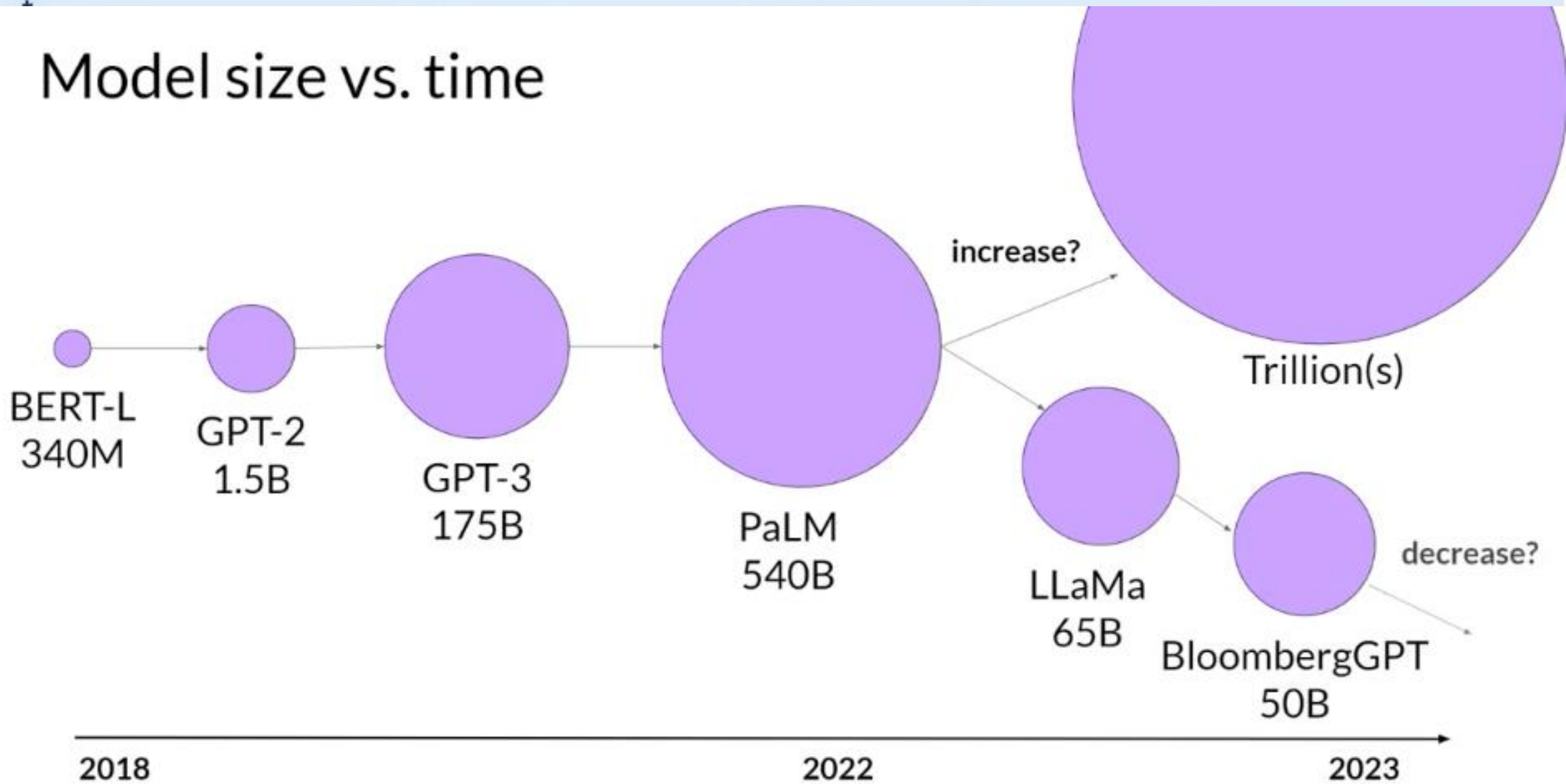
- A. Batch size: Number of samples per iteration
- B. Model size: Number of parameters
- C. Compute budget: Compute constraints
- D. Dataset size: Number of tokens



Model size vs. time



Model size vs. time



Pre-training for domain adaptation





Pre-training for domain adaptation

Legal language

The prosecutor had difficulty proving mens rea, as the defendant seemed unaware that his actions were illegal.

The judge dismissed the case, citing the principle of res judicata as the issue had already been decided in a previous trial.

Despite the signed agreement, the contract was invalid as there was no consideration exchanged between the parties.

Medical language

After a strenuous workout, the patient experienced severe myalgia that lasted for several days.

After the biopsy, the doctor confirmed that the tumor was malignant and recommended immediate treatment.

Sig: 1 tab po qid pc & hs



Take one tablet by mouth four times a day, after meals, and at bedtime.



BloombergGPT: domain adaptation for finance

BloombergGPT: A Large Language Model for Finance

Shijie Wu^{1,*}, Ozan İrsoy^{1,*}, Steven Lu^{1,*}, Vadim Dabravolski¹, Mark Dredze^{1,3}, Sebastian Gehrmann¹, Prabhanjan Kambadur¹, David Rosenberg², Gideon Mann¹

¹ Bloomberg, New York, NY USA

² Bloomberg, Toronto, ON Canada

³ Computer Science, Johns Hopkins University, Baltimore, MD USA

~51%

**Financial
(Public & Private)**

Abstract

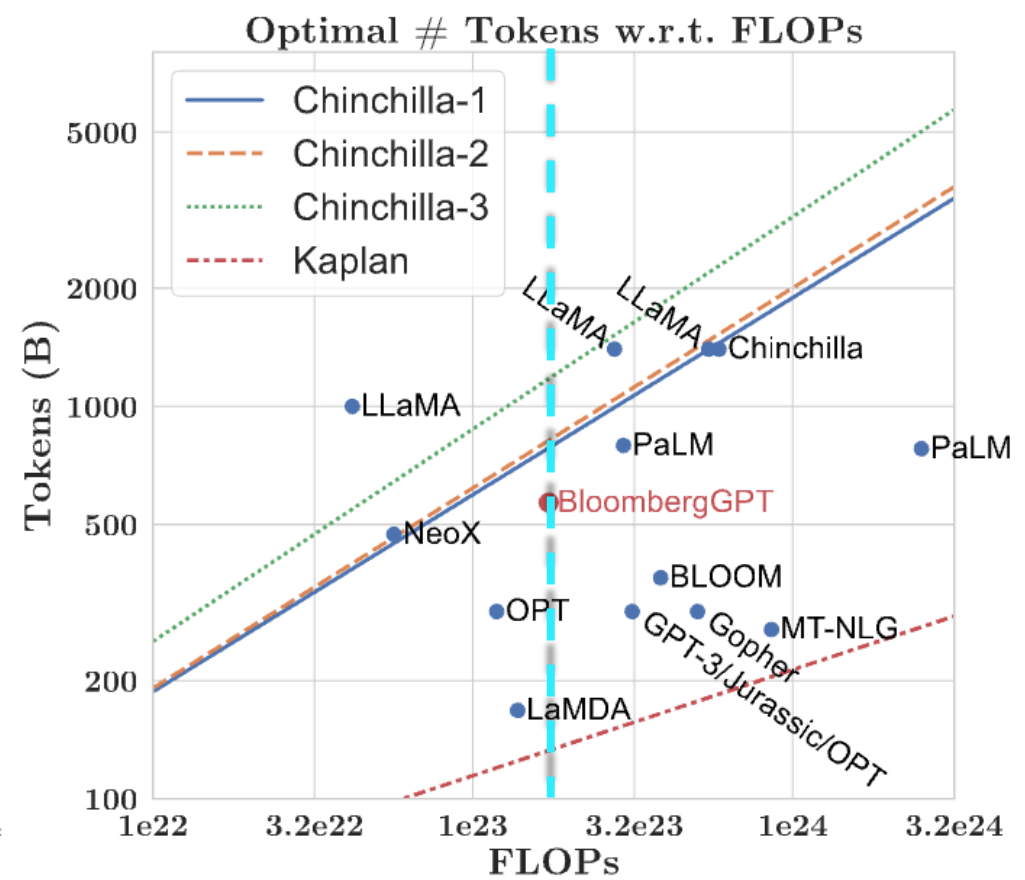
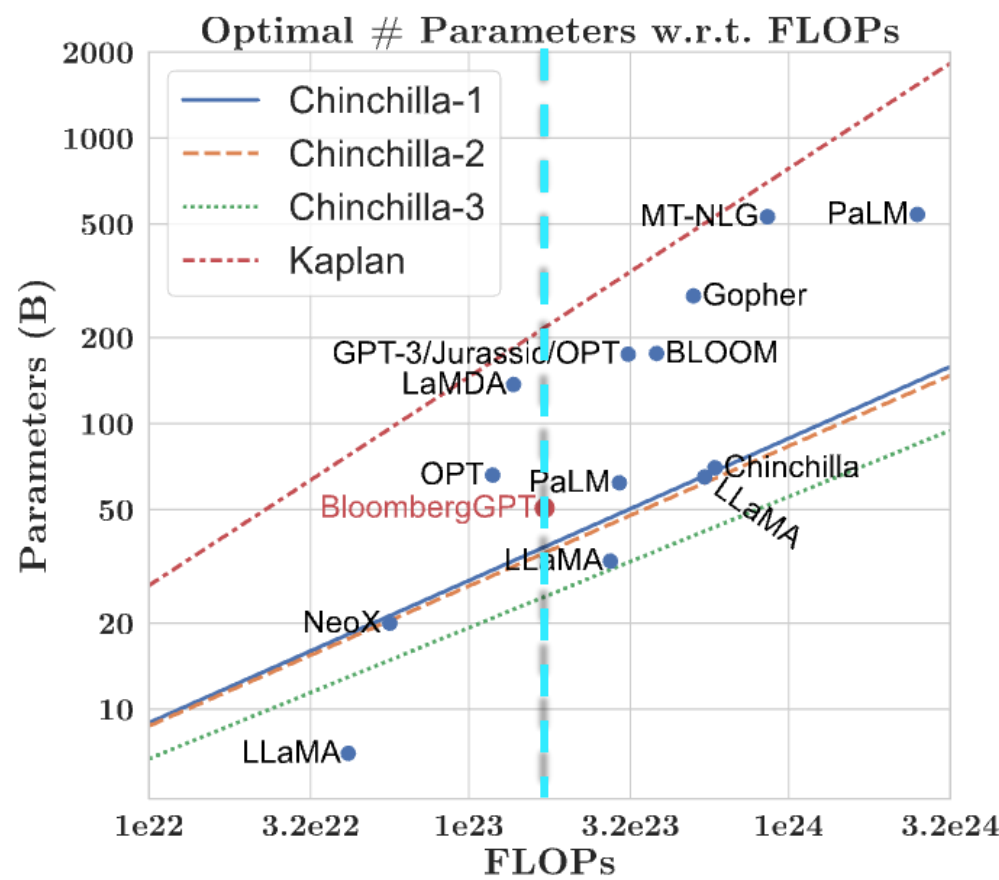
The use of NLP in the realm of financial technology is broad and complex, with applications ranging from sentiment analysis and named entity recognition to question answering. Large Language Models (LLMs) have been shown to be effective on a variety of tasks; however, no LLM specialized for the financial domain has been reported in literature. In this work, we present BLOOMBERGGPT, a 50 billion parameter language model that is trained on a wide range of financial data. We construct a 363 billion token dataset based on Bloomberg's extensive data sources, perhaps the largest domain-specific dataset yet, augmented with 345 billion tokens from general purpose datasets. We validate BLOOMBERGGPT on standard LLM benchmarks, open financial benchmarks, and a suite of internal benchmarks that most accurately reflect our intended usage. Our mixed dataset training leads to a model that outperforms existing models on financial tasks by significant margins without sacrificing performance on general LLM benchmarks. Additionally, we explain our modeling choices, training process, and evaluation methodology. We release Training Chronicles ([Appendix C](#)) detailing our experience in training BLOOMBERGGPT.

~49%

**Other
(Public)**



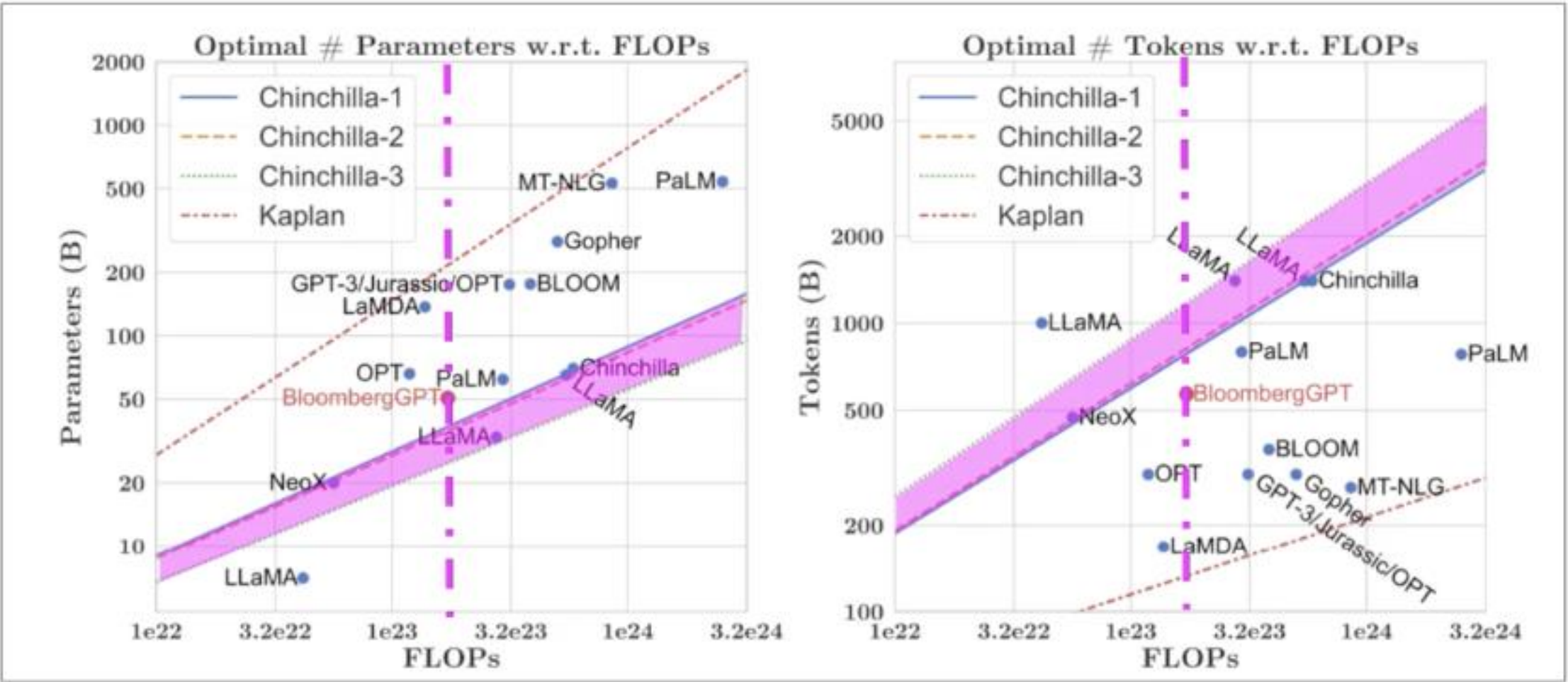
BloombergGPT relative to other LLMs



Source: Wu et al. 2023, "BloombergGPT: A Large Language Model for Finance."

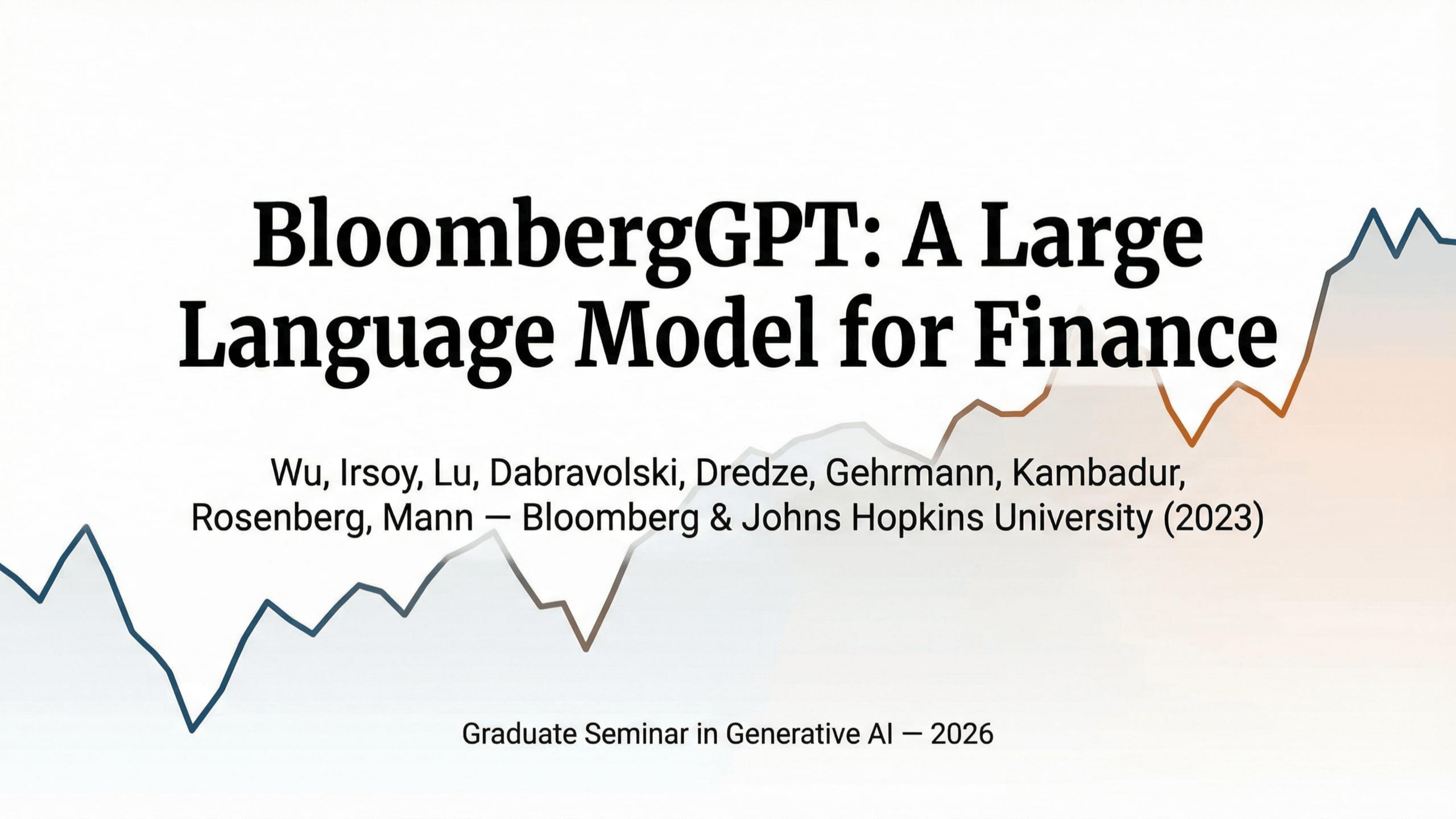


BloombergGPT relative to other LLMs



Source: Wu et al. 2023, "BloombergGPT: A Large Language Model for Finance"

BloombergGPT: A Large Language Model for Finance

The background features a stylized line graph with a blue line and an orange line, both showing fluctuations. The area under the blue line is shaded light blue, and the area under the orange line is shaded light orange. The lines and shaded areas extend across the width of the slide, with the orange line appearing more prominent on the right side.

Wu, Irsoy, Lu, Dabravolski, Dredze, Gehrmann, Kambadur,
Rosenberg, Mann — Bloomberg & Johns Hopkins University (2023)

Why Build a Domain-Specific LLM for Finance?

- Financial NLP is broad and complex: sentiment analysis, NER, question answering, news classification.
- General LLMs (GPT-3, PaLM, BLOOM) cover many domains but lack financial specialization.
- Financial language has unique characteristics: Domain-specific terminology (tickers, filings, BQL, derivatives jargon) and numerical reasoning over financial tables.
- Prior domain-specific models existed for science (Galactica), medicine (BioGPT), but NONE for finance.
- Masked LMs existed (FinBERT) but no generative/autoregressive LLM for finance.

Sentiment Analysis: General vs. Financial Context	
General Sentiment	Financial Sentiment
 "COMPANY to cut 10,000 jobs" Negative	 "COMPANY to cut 10,000 jobs" Positive (cost reduction)

Approaches to Domain-Specific LLMs: BloombergGPT's Novel Strategy

General-purpose only

Train on broad web data (e.g., GPT-3, PaLM).

Limitation: Poor on specialized tasks.

Domain-specific only

Train exclusively on domain data (e.g., Galactica).

Limitation: Loses general capabilities.

Mixed (BloombergGPT)

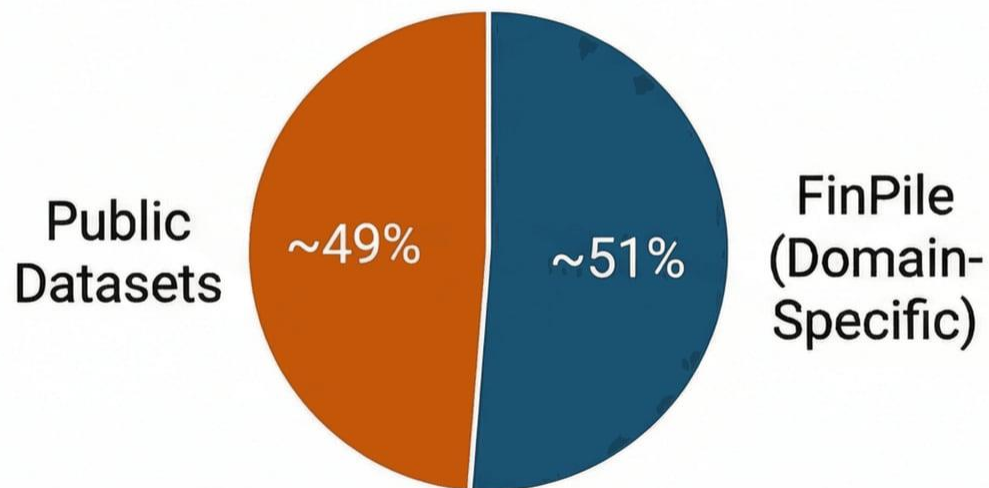
Train on **both** domain + general data (~50/50 split).

Goal: Best-in-class on financial tasks **WITHOUT** sacrificing general NLP performance.



FinPile: Constructing the Largest Domain-Specific Dataset (363B Tokens)

- Total training corpus: **~709 billion tokens** (569B used for training)
- FinPile (domain-specific): **363B tokens (51.27%)**
- Public datasets: **345B tokens (48.73%)**
- Documents time-stamped from 2007-03 to 2022-07
- Bloomberg's **40 years** of curated financial data archives



FinPile Breakdown		
Source	Tokens	Percentage
Web	298B	42.01%
News	38B	5.31%
Filings	14B	2.04%
Press	9B	1.21%
Bloomberg	5B	0.70%

Public Data Sources and the Unigram Tokenizer

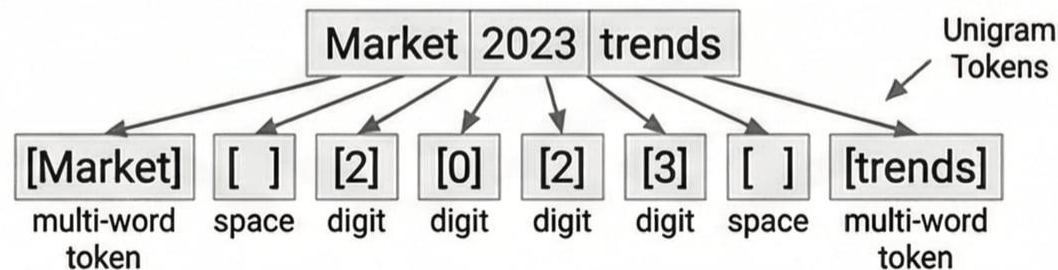
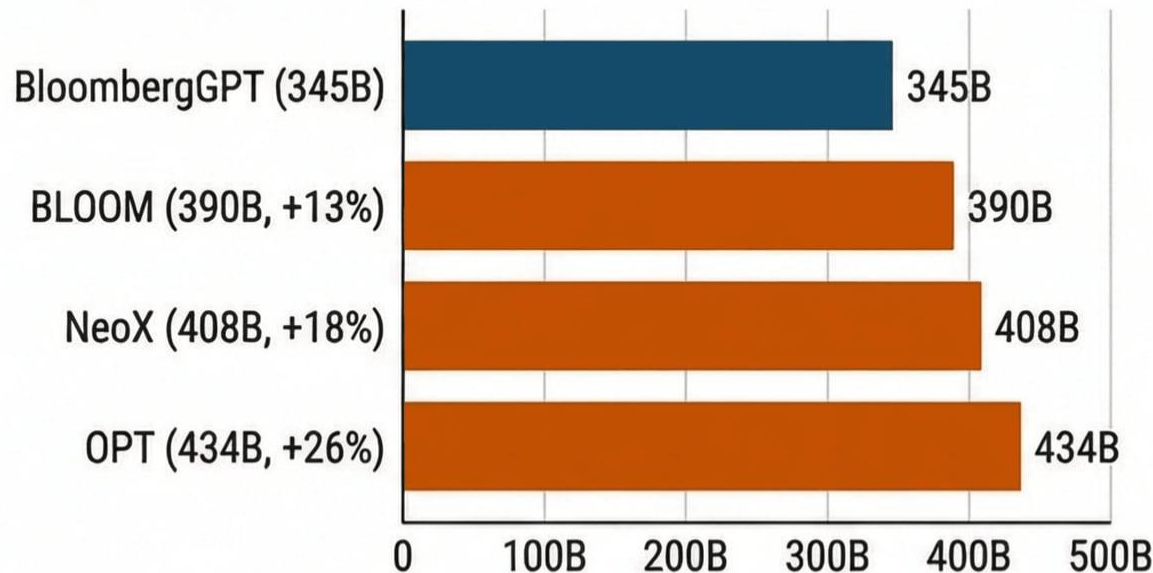
Public Datasets (345B tokens, 48.73%)

- The Pile (184B)
- C4 (138B)
- Wikipedia (24B)

Unigram Tokenizer

- Tokenization Choice: Unigram tokenizer instead of standard BPE.
- Vocabulary Size: 131,072 tokens (2^{17}).
- Advantages: Saves probabilities, allows multi-word tokens, each digit gets its own token.

Tokenizer Efficiency on Public Data (Tokens, Billions)



Architecture: A 50.6B Parameter BLOOM-Style Decoder

Key Design Choices

- Decoder-only causal language model based on BLOOM
- ALiBi positional encoding (no explicit positional embeddings)
- GELU activation in FFN
- Additional LayerNorm after token embeddings
- Input token embeddings tied to output linear mapping

Formulas

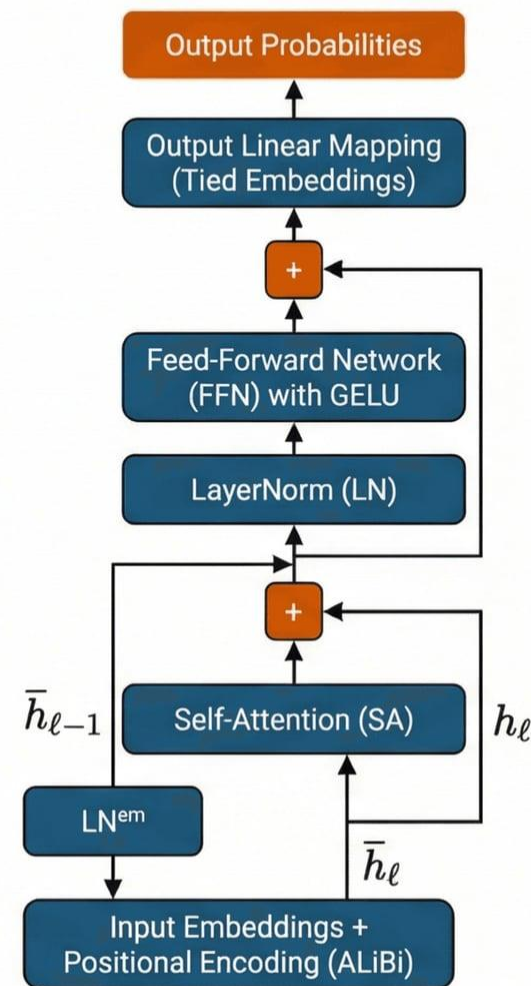
$$\bar{h}_\ell = h_{\ell-1} + \text{SA}(\text{LN}(h_{\ell-1}))$$

$$h_\ell = \bar{h}_\ell + \text{FFN}(\text{LN}(\bar{h}_\ell))$$

$$\bar{h}_1 = \text{LN}^{\text{em}}(h_0) + \text{SA}(\text{LN}(\text{LN}^{\text{em}}(h_0)))$$

Model Configuration

Parameter	Value
Layers	70
Heads	40
Hidden Dim	7,680
Vocab Size	131,072
Total Params	50.6B
Hardware	512 A100 40GB GPUs



Model Sizing via Chinchilla Scaling Laws

Key Points

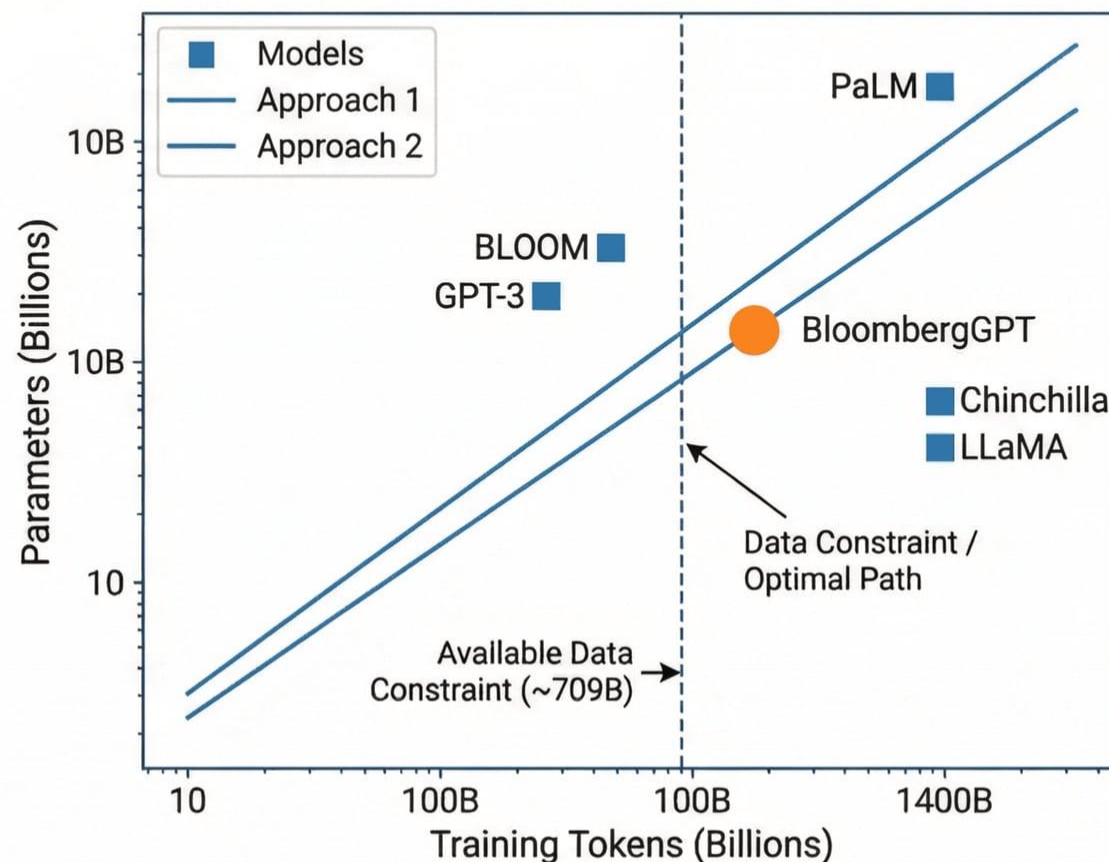
- Compute Budget: 1.3M GPU hours on A100 40GB GPUs (effective budget = $0.75 \times 1.3\text{M}$ due to activation checkpointing)
- The Data Constraint Problem: Chinchilla-optimal for 50B params needs $\sim 1,100\text{B}$ tokens, but available data is only $\sim 709\text{B}$ tokens.
- Cannot increase domain data (FinPile already the largest domain-specific dataset) and do not want domain data $< 50\%$ of total training.
- Solution: Train 50B model on 569B tokens with 30% compute buffer for failures.

Chinchilla Scaling Calculations

Approach 1: $\text{Parameters} = \exp(\log_{10}(\text{FLOPs}) \times 0.498 - 1.004)$
= 52.99B | Tokens = 1,111B

Approach 2: $\text{Parameters} = \exp(\log_{10}(\text{FLOPs}) \times 0.490 - 0.839)$
= 49.75B | Tokens = 1,176B

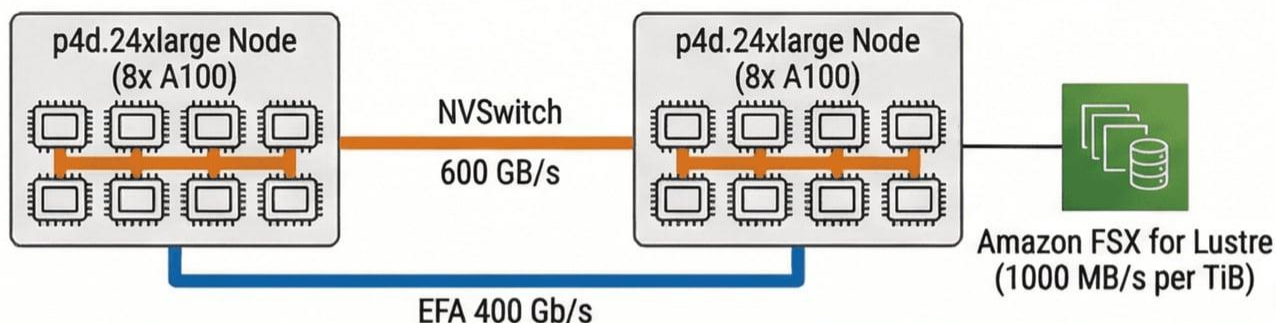
Chinchilla Scaling Laws & BloombergGPT



Large-Scale Training: Infrastructure and Optimization

Hardware Stack

- Amazon SageMaker on AWS
- 64 × p4d.24xlarge instances (8 × A100 40GB each = 512 GPUs)
- NVSwitch intra-node: 600 GB/s | EFA inter-node: 400 Gb/s
- Amazon FSX for Lustre: up to 1000 MB/s read/write per TiB



Key Numerical Details

- Batch size warm-up: 1,024 for 7,200 steps → 2,048
- Total training: 139,200 steps (~53 days)
- Trained on 569B tokens out of 709B available (80% of one epoch)
- Total FLOPs: 2.36×10^{23}

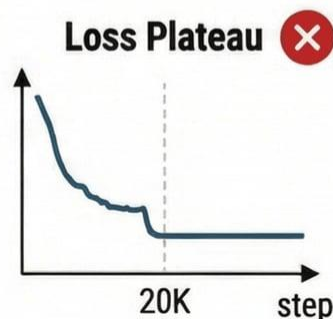
Optimization Techniques

Technique	Description / Benefit
ZeRO Stage 3	Shard parameters, gradients, optimizer state across 128 GPUs; 4 model copies
MiCS	Hierarchical communication, 2-hop gradient update for cloud clusters
Activation Checkpointing	Discard intermediate activations; recompute during backward pass
Mixed Precision	Forward/backward in BF16; parameters stored/updated in FP32
Fused Kernels	Masked-causal-softmax fused kernel; 4-5 TFLOPs improvement

Training Chronicles: Four Attempts and Hard-Won Lessons

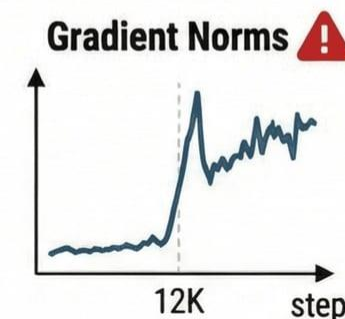
Run v0 (Failed)

- **Curriculum Learning.** Temporally ordered data (2007->2022).
- Model stuck after ~20K steps.



Run v1.x (Failed)

- **The “Elbow” Problem.** Gradient norms increased after ~12K steps.
- **Root cause:** Input LayerNorm at Layer 1 elbowed upward.
- **Bug:** weight decay on LayerNorm multipliers.



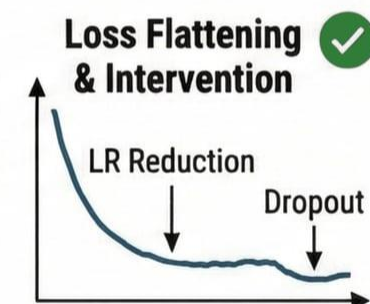
Run v2.0 (Success)

- **Conservative Restart.** Started from scratch.
- Lower LR ($6e-5$), tighter gradient clip (0.3), added LN^{em} , fixed weight decay bug.
- Smooth training for 42 days.



Run v2.x: Late-Stage Interventions

- Validation loss **flattened**. Reduced LR, added dropout (0.1).
- Final checkpoint at step 139,200.



Comprehensive Evaluation: Financial + General Benchmarks

59 total tasks across six categories.

Few-shot Methodology: No prompt tuning, no Chain-of-Thought – “standard” prompting only.

Methods: Regular, Calibration, Normalization (report best per model per task).

Evaluation Benchmarks

Benchmark Category	Number of Tasks
Public Financial Tasks	5
Bloomberg Financial Tasks	12
BIG-bench Hard	23
Knowledge Assessments	5
Reading Comprehension	5
Linguistic Tasks	9

Comparison Models

Model	Parameters (B)	Compute (x)
BloombergGPT	50.6B	1.00x
GPT-NeoX	20B	0.33x
OPT-66B	66B	0.69x
BLOOM-176B	176B	2.24x
GPT-3	175B	1.82x

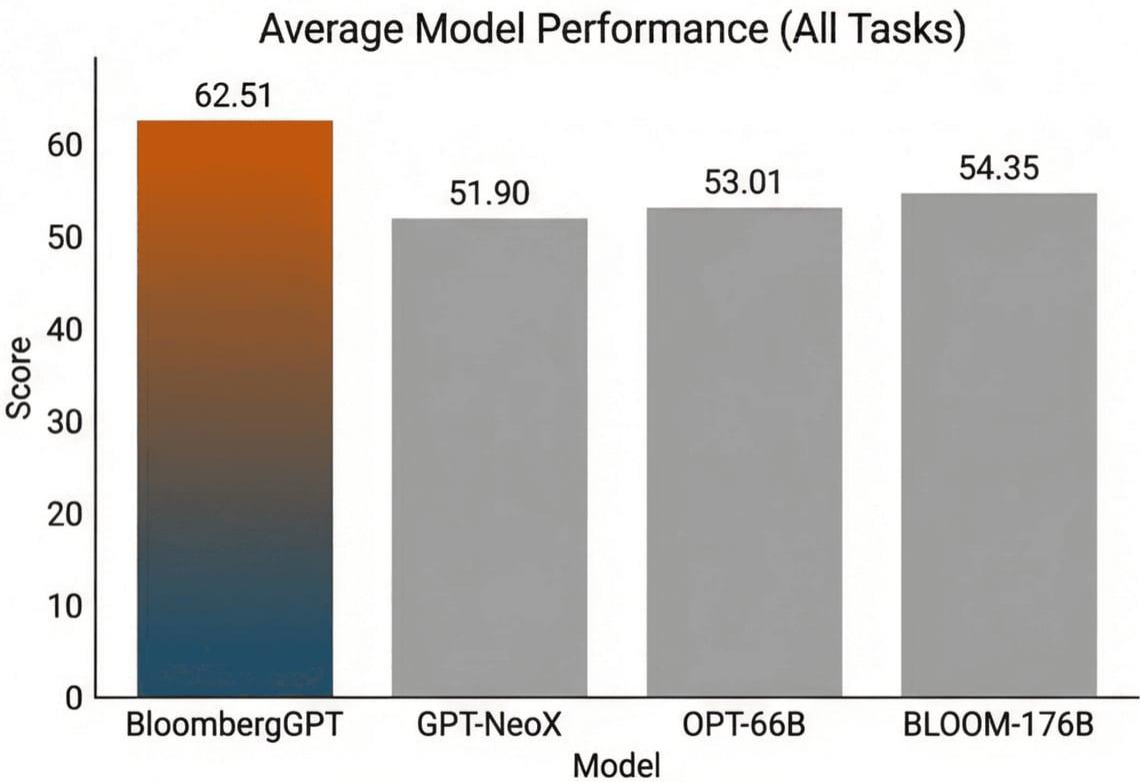
Few-shot Methodology

- **Regular:** Standard few-shot prompting. $\text{Score} = P(\text{response} \mid \text{context, examples})$
- **Calibration:** Regular score adjusted by pre-computed bias. $\text{Score} = P(\text{response} \mid \text{context, examples}) / P(\text{response} \mid \text{context, no examples})$
- **Normalization:** Regular score adjusted by score distribution. $\text{Score} = (P(\text{response} \mid \text{context, examples}) - \text{mean}(P)) / \text{std}(P)$

Financial Tasks: BloombergGPT Dominates

Key Points

- **BloombergGPT** wins **4 of 5** external financial tasks.
- Largest gap on **ConvFinQA** (+7.1 over BLOOM-176B), requiring numerical reasoning over financial tables.
- Win Rate: **0.93** vs **0.47** (BLOOM-176B).



External Financial Task Results				
Task	BloombergGPT	GPT-NeoX	OPT-66B	BLOOM-176B
ConvFinQA	43.41	30.06	27.88	36.31
FiQA SA	75.07	50.59	51.60	53.12
FPB	51.07	44.64	48.67	50.25
Headline	82.20	73.22	79.41	76.51
NER	60.82	60.98	57.49	55.56
All Tasks (avg)	62.51	51.90	53.01	54.35

Internal Sentiment Analysis: Massive Margins of Victory

Key Points

- **BloombergGPT** achieves a **PERFECT** win rate of **1.00** on internal tasks.
- Equity News: **79.63** vs. next best **20.98** (OPT) – a **59-point gap**.
- Country News: **49.14** vs. next best **17.41** (OPT) – a **32-point gap**.

These are aspect-specific sentiment tasks requiring deep financial domain understanding.

1.00 WIN RATE
PERFECT PERFORMANCE
59-POINT GAP
EQUITY NEWS ADVANTAGE

Bloomberg Internal Sentiment Tasks Results

Task	BloombergGPT	GPT-NeoX	OPT-66B	BLOOM-176B
Equity News	79.63	14.17	20.98	19.96
Equity Social Media	72.40	66.48	71.36	68.04
Equity Transcript	65.06	25.08	37.58	34.82
ES News	46.12	26.99	31.44	28.07
Country News	49.14	13.45	17.41	16.06
All Tasks (avg)	62.47	29.23	35.76	33.39

Exploratory Tasks: NER and Named Entity Disambiguation

Why NER is Hard for Generative LLMs

- Information extraction task (better for encoders)
- Requires extensive prompt engineering (20-shot)

Finance-Specific Subtleties

- “Bloomberg” -> reporting org or entity?
- “Twitter” -> ORG or product?

NER+NED Example

Input: “AAPL announced that they will stop using Intel chips”

NER+NED Output: AAPL, INTC
(requires knowledge of company-to-ticker mappings)

NER Results

Model	Win Rate
BLOOM-176B	0.95

NER+NED Results

Model	Win Rate	Avg Score
BloombergGPT	0.95	64.83
BLOOM-176B	-	45.43

General NLP: Competitive Despite Domain Focus

Despite its domain-specific training, BloombergGPT demonstrates impressive performance across general Natural Language Processing benchmarks.

BIG-bench Hard

Model	Score
BloombergGPT	41.97 ✓
BLOOM-176B	44.91

BloombergGPT is among top performers in its size class.

Knowledge Assessments

Metric	Result
BloombergGPT Win Rate	0.75

High win rate indicates strong factual knowledge.

Reading Comprehension

Model	Score
BloombergGPT	61.22
BLOOM-176B	49.37

Significantly outperforms BLOOM-176B on comprehension tasks.

Linguistic Tasks

Model	Score
BloombergGPT	60.63
BLOOM-176B	58.26

Demonstrates competitive linguistic capabilities.

Key Takeaway: BloombergGPT consistently ranks among the **top-2 models** in its parameter class and frequently surpasses BLOOM-176B (3.5x larger), showcasing its competitive general NLP capabilities.

Qualitative Demonstrations: From BQL to CEO Knowledge

Exploring Generative AI Capabilities in Finance

Bloomberg Query Language (BQL) Generation

Input: `Get me the last price and market cap for Apple`

Output: `get(px_last, cur_mkt_cap) for(['AAPL US Equity'])`

News Headline Suggestion

Input: `[Article about US housing market dropping $2.3 trillion]`

Output: `"Home Prices See Biggest Drop in 15 Years"`

Financial Knowledge — CEO Identification

Company & Model Comparison	BloombergGPT (Correct)	GPT-NeoX (Comparison)
CEO of Citigroup Inc?	Jane Fraser ✓ (Correct)	Michael L Corbat ✗ (Outdated)
CEO of Silicon Valley Bank?	Greg Becker ✓ (Correct)	John Hodor ✗ (Incorrect)

Conclusions, Limitations, and the Legacy of BloombergGPT

Contributions

- First large-scale domain-specific LLM for finance (50.6B params).
- Demonstrated mixed training (domain + general data) achieves best of both worlds.
- Constructed FinPile (363B tokens) and provided detailed Training Chronicles.

Limitations

- Model not publicly released; FinPile not reproducible.
- Somewhat “undertrained” by Chinchilla standards.
- No instruction tuning or RLHF alignment explored.

Historical Impact (2026 Perspective)

- Cited 1,730+ times; inspired FinGPT and open-source financial LLMs.
- Mixed-training paradigm became the standard approach.

Open Questions for Discussion

- Is domain-specific pre-training still necessary in 2026 with GPT-4/Claude/Gemini?
- Could fine-tuning or RAG achieve similar results more efficiently?